

T13-RR4 TR4-G4M3

Integrantes

Jorge Moya Lucas 56971

Marcos Rodríguez Pena 56987

Alba Martínez Fernández 56961

Miguel Olalla Taladriz 56978

Paula Verdejo Rodríguez 57000



ÍNDICE GENERAL

1.Introducción.....	3
1.1 Breve descripción del proyecto.....	4
1.2 Objetivos del robot.....	4
2. Actualidad.....	5
3. Diseño Conceptual.....	6
4. Diseño estructural y matemático.....	8
4.1 Detalles del diseño de la estructura del robot.....	8
4.2 Primeras versiones del diseño 3D.....	10
4.3 Modelo final.....	14
4.3.1 SWIVEL.....	14
4.3.2 Pitch.....	17
4.3.3 Elbow.....	18
4.3.4 Pala.....	19
4.4 Cinemática.....	21
5. DISEÑO ELECTRÓNICO.....	26
5.1 Componentes electrónicos.....	26
5.1.1 Motor corriente continua con encoder.....	26
5.1.2 L298N - Puente H.....	28
5.1.3 ESP32-WROOM-32.....	30
5.1.4 PCB.....	31
5.2 Implementación de los motores.....	35
5.2.1 Pruebas iniciales y conexiones.....	35
5.2.2 Realimentación.....	35
5.2.3 PID.....	36
6. APLICACIÓN.....	41
6.1. Descripción general.....	41
6.2. Funcionalidades.....	41
6.3.Tecnologías utilizadas.....	41
6.4. Códigos implementados.....	42
7. Fases del montaje.....	57
8. Software.....	63
9. Presupuesto.....	64
10. Resultado.....	65
11. Reparto de tareas.....	67
12. NOTAS.....	68
13. CÓDIGO FINAL.....	68

1.Introducción

Este proyecto ha supuesto un auténtico desafío técnico y creativo dentro del ámbito de la robótica, ya que nos ha llevado a diseñar, construir y controlar un brazo robótico desde cero, con la única ayuda de nuestras competencias adquiridas durante el grado y un presupuesto limitado. Lejos de seguir un modelo prefabricado, hemos afrontado todas las fases del desarrollo de manera integral: desde el diseño mecánico hasta la programación del sistema de control, pasando por la creación de una interfaz personalizada para su manejo.

La tarea principal que debía desempeñar nuestro robot consistía en retirar arena mojada de una superficie plana definida y depositarla en otra zona, generando un montículo. Esta acción, aparentemente sencilla, encierra una notable complejidad técnica, sobre todo si se realiza con un brazo robótico no cartesiano y con un diseño lo suficientemente robusto como para trabajar con materiales húmedos, pesados y con comportamiento semisólido.

Dado el contexto del ejercicio —trabajo estacionario en un espacio acotado y manipulación de cargas variables— se presupone que el brazo debía estar anclado de forma fija al suelo, como ocurre en muchas soluciones reales de robótica industrial. Esta decisión de diseño nos permitió concentrar los grados de libertad en el brazo y optimizar tanto la estabilidad como la precisión del sistema.

Desde el inicio, decidimos inspirarnos en la morfología de una excavadora tradicional, adaptándola al entorno robótico y a las restricciones de espacio. Esta referencia nos sirvió como base para dotar al brazo de una herramienta eficaz en la recogida y transporte de materiales granulados como la arena húmeda, que tiende a compactarse y adherirse a las superficies. Diseñamos un extremo operativo específico para esta tarea, asegurando la funcionalidad incluso en condiciones de trabajo poco ideales.

Uno de los aspectos más destacables del proyecto ha sido la autonomía completa en el diseño mecánico: todo el sistema ha sido modelado desde cero en Fusion 360, sin recurrir a diseños externos. Esto nos permitió optimizar dimensiones, pesos, geometrías y puntos de articulación en función de los componentes reales y de nuestras necesidades. El resultado es un brazo equilibrado, funcional y eficiente, construido con materiales accesibles pero con un diseño plenamente profesional.

Como complemento a la parte física, desarrollamos una aplicación móvil personalizada para el control remoto del robot. Esta interfaz mejora notablemente la experiencia de usuario y ofrece una vía práctica para realizar pruebas, ajustar trayectorias o incluso implementar automatizaciones en el futuro.

Durante el proceso, nos enfrentamos a numerosos retos técnicos: desde la selección adecuada de motores con el par necesario hasta la implementación del control en lazo

cerrado en varios ejes. Cada uno de estos desafíos nos obligó a poner en práctica conocimientos de distintas asignaturas del grado, como control automático, cinemática, programación embebida, diseño asistido por ordenador e instrumentación.

Este trabajo ha supuesto una oportunidad única para integrar disciplinas, enfrentarnos a problemas reales de ingeniería y crear una solución funcional que cumple con todos los requisitos propuestos. El resultado final es un brazo robótico de diseño propio, inspirado en maquinaria real, capaz de recoger arena mojada con precisión, controlado por una app móvil y con todos sus sistemas desarrollados desde cero.

1.1 Breve descripción del proyecto

El proyecto consiste en el diseño, construcción y control de un brazo robótico inspirado en la morfología de una excavadora tradicional, con la particularidad de estar anclado a un punto fijo. Todo el diseño ha sido realizado íntegramente desde cero utilizando la herramienta de CAD Autodesk Fusion 360, incluyendo tanto la estructura mecánica como los elementos de transmisión, alojamientos para motores, sensores y el diseño del extremo operativo.

El brazo robótico está compuesto por varios eslabones articulados que le otorgan múltiples grados de libertad, permitiendo alcanzar de forma precisa zonas de difícil acceso, incluidas esquinas o rincones del área de trabajo. Se ha diseñado para manipular arena mojada, un material de alta densidad y baja fluidez, lo que añade una complejidad importante al proceso de recolección y transporte de carga, en comparación con materiales secos o ligeros.

El sistema incorpora un controlador basado en microcontroladores ESP32, que permite tanto la ejecución de algoritmos de control en tiempo real como la comunicación inalámbrica con una aplicación móvil propia, desarrollada específicamente para este proyecto. Esta aplicación permite al usuario controlar el movimiento del brazo de forma intuitiva, así como supervisar en tiempo real el estado del sistema, aportando un nivel de interacción avanzado y profesional.

El robot integra también sensores externos, con el objetivo de mejorar la precisión y autonomía del sistema. Por ejemplo, sensores de distancia o de fuerza pueden emplearse para estimar la cantidad de arena recogida o evitar colisiones. Estos sensores constituyen una mejora significativa frente a un sistema puramente manual o abierto.

1.2 Objetivos del robot

Los objetivos principales que guían el desarrollo del proyecto se detallan a continuación:

1. Diseñar un brazo robótico no cartesiano desde cero, utilizando herramientas profesionales de modelado 3D (Fusion 360), con una estructura robusta y

funcional que permita la manipulación de arena mojada en condiciones reales.

2. Implementar un sistema de control en lazo cerrado para al menos dos grados de libertad del brazo, incorporando sensores y desarrollando el lazo completo (actuador – sensor – controlador), cumpliendo con los requisitos obligatorios del proyecto.
3. Desarrollar una interfaz de usuario móvil personalizada, que permita el control del robot de forma remota e intuitiva, mejorando la experiencia de usuario y facilitando la operación del sistema.
4. Incorporar sensorización inteligente para optimizar la ejecución de la tarea, como estimación de carga, detección de obstáculos o delimitación del área de trabajo.
5. Conseguir una manipulación efectiva de arena mojada, valorando especialmente las soluciones creativas para la recolección y vertido del material, así como la capacidad del robot para acceder a zonas difíciles.
6. Respetar una limitación presupuestaria inferior a 200€, lo que implica seleccionar cuidadosamente los componentes mecánicos, electrónicos y estructurales, buscando el mejor equilibrio entre coste, rendimiento y fiabilidad.
7. Fomentar el carácter integrador del trabajo, incluyendo conocimientos de múltiples áreas del grado, desde programación de sistemas embebidos hasta control automático, instrumentación, modelado cinemático y diseño CAD.

2. Actualidad

Robot similar en el mercado

En la actualidad, la robótica aplicada a la manipulación de materiales sueltos —como arena, grava o tierra— está ampliamente desarrollada en sectores como la construcción, la minería y la logística. Existen numerosos brazos robóticos diseñados específicamente para la manipulación de materiales pesados o difíciles de controlar, como la arena mojada, que presenta características especiales debido a su cohesión, humedad y peso variable.

Un ejemplo claro de robot similar en el mercado es el robot manipulador hidráulico montado sobre base fija, utilizado en plantas de reciclaje, minería o líneas de carga automatizadas. Estos brazos, como los fabricados por Brokk, KUKA (con adaptaciones especiales) o FANUC, están diseñados para operaciones estacionarias, con múltiples grados de libertad y una pinza o cuchara adaptada para remover y depositar materiales pesados. Aunque su precio y complejidad los alejan del ámbito educativo o amateur,

comparten la esencia funcional de nuestro proyecto: retirar material granular de una superficie y transportarlo a otra zona definida del entorno de trabajo.

Por ejemplo, la serie KUKA KR QUANTEC, aunque diseñada para tareas industriales de alta precisión, se puede adaptar para manipular materiales con una herramienta apropiada en su extremo operativo. Estos robots cuentan con alta capacidad de carga, estructura no cartesiana y gran repetibilidad. En nuestro caso, hemos recreado una versión simplificada de este tipo de sistema, utilizando elementos de bajo coste y con una funcionalidad inspirada en una excavadora fija, que representa una solución similar pero más accesible.

Otra referencia relevante son los robots de movimiento articulado utilizados en procesos agrícolas automatizados, como los brazos robóticos diseñados para manipular tierra o sustrato en cultivos verticales. Algunos modelos, como los desarrollados por Octinion o Agrobot, presentan estructuras similares a la de nuestro diseño: anclaje al suelo, brazo articulado, movimientos suaves y herramientas especializadas en la manipulación de materiales no uniformes. Estos robots también integran sensores para adaptar sus acciones a las condiciones del entorno, lo cual está alineado con el enfoque de mejora que nosotros hemos seguido al incluir control en lazo cerrado y capacidad de integración de sensores.

Cabe destacar que, aunque existen sistemas industriales complejos y muy potentes para tareas similares, no es habitual encontrar en el mercado una solución de bajo coste, programable con software libre y con diseño completamente abierto como la que hemos desarrollado nosotros. Nuestro brazo robótico representa una versión funcional, educativa y modular de estos grandes sistemas, adaptada a un entorno académico pero sin renunciar a los principios fundamentales de la robótica profesional.

Así, nuestro proyecto se sitúa como una propuesta realista y asequible inspirada en soluciones industriales, pero adaptada al entorno universitario con medios limitados. A pesar de su sencillez relativa, incluye conceptos esenciales como la cinemática directa e inversa, control por sensores, comunicación remota y autonomía parcial, lo que lo convierte en una valiosa aproximación al mundo real de la robótica manipuladora.

3. Diseño Conceptual

Concepto del robot

Desde el inicio del proyecto se apostó por una solución no cartesiana que respondiera a los requisitos establecidos en la asignatura, pero que además ofreciera una aproximación realista y funcional al problema de manipular arena húmeda. Inspirándonos en el funcionamiento de una excavadora tradicional, optamos por desarrollar un brazo robótico de morfología articulada, anclado a una base fija, capaz de simular los movimientos de un brazo de carga mecánico mediante sistemas de transmisión por correas.

La elección de esta configuración no fue aleatoria: las excavadoras son máquinas probadas en el trabajo con materiales granulares o cohesivos, como la arena mojada, y su geometría permite una combinación eficaz de alcance, fuerza y simplicidad estructural. Reinterpretamos ese diseño en clave robótica, manteniendo el enfoque sobre la funcionalidad y la estabilidad del sistema.

Todo el diseño ha sido realizado por completo por nuestro equipo, desde cero, utilizando el software de modelado paramétrico Fusion 360. Esta herramienta nos ha permitido desarrollar un diseño modular, preciso y optimizado tanto para impresión 3D como para integración con componentes electrónicos y mecánicos reales. El proceso de diseño ha abarcado desde la ideación de la geometría de los eslabones hasta los sistemas de anclaje, rodamientos y transmisión.

Uno de los aspectos más destacables de nuestra propuesta es el diseño original de la base rotatoria del brazo, un componente que ha sido completamente desarrollado por nosotros y que introduce una solución ingeniosa y funcional. La base está constituida por un cilindro vertical anclado al suelo, que actúa como carcasa externa y soporte estructural. Este cilindro tiene un orificio interno que permite la conexión mecánica entre un motor externo y una base giratoria interna mediante correas de transmisión. Esta base giratoria está montada sobre un rodamiento de gran diámetro, lo que permite que toda la estructura superior del brazo (los eslabones articulados y la pala) pueda rotar libremente en el plano horizontal, mientras que el cilindro exterior permanece completamente rígido.

Este sistema permite separar claramente la función de giro horizontal del resto de grados de libertad del brazo, simplificando el control y mejorando la estabilidad. Además, el uso de correas en lugar de engranajes u otros métodos de transmisión mecánica permite reducir el coste y facilitar el mantenimiento, sin sacrificar precisión.

Elección de materiales

En cuanto a los materiales empleados para la construcción del brazo robótico, se ha apostado por una combinación de componentes metálicos y piezas fabricadas mediante impresión 3D, buscando el equilibrio entre resistencia, precisión, estética industrial y facilidad de integración.

Las piezas estructurales diseñadas por nuestro equipo han sido impresas en una impresora de resina, la cual estaba disponible dentro del grupo. Esta tecnología de impresión nos ha permitido obtener piezas con altísima precisión y excelente acabado superficial, fundamentales para el correcto alojamiento de los rodamientos y el montaje de las articulaciones. La rigidez y estabilidad dimensional que ofrece la resina es superior a la del PLA o el ABS tradicionales, lo que ha mejorado el comportamiento mecánico general del sistema, especialmente en las zonas que requieren un ajuste fino o que soportan esfuerzos laterales.

Para los eslabones del brazo, hemos empleado tubos cuadrados de aluminio, un material ligero pero resistente que aporta un toque más profesional e industrial al diseño. El aluminio no solo garantiza una buena durabilidad frente a las exigencias

mecánicas del sistema, sino que también facilita el montaje de motores y poleas gracias a su facilidad de mecanizado y perforado.

El resto de componentes mecánicos y electrónicos utilizados en el proyecto son elementos comerciales estándar y fácilmente disponibles. Entre ellos se incluyen:

Rodamientos de bolas (como los 608ZZ) para permitir un movimiento suave en las articulaciones.

Correas dentadas y poleas, encargadas de transmitir el movimiento de los motores a los diferentes grados de libertad del brazo.

Motores eléctricos.

Elementos electrónicos necesarios para la sensorización, el control y la comunicación del sistema (placa Arduino/ESP32, drivers de motor, cables, etc.).

Esta selección de materiales ha permitido construir un prototipo robusto, funcional y dentro de los límites de presupuesto establecidos, sin comprometer la calidad técnica ni el potencial de mejora del sistema.

4. Diseño estructural y matemático

4.1 Detalles del diseño de la estructura del robot

Base

La base del brazo robótico es el componente fundamental sobre el que se articula toda la estructura. En nuestro diseño, se ha optado por una estructura cilíndrica fija anclada al suelo, cumpliendo con el requisito implícito de que el brazo esté anclado a un punto fijo. Esta base no solo proporciona estabilidad estructural y resistencia frente a los esfuerzos de torsión y flexión generados por los movimientos del brazo, sino que también actúa como soporte del sistema de giro horizontal del conjunto superior.

El cuerpo cilíndrico contiene en su interior un conjunto de sistemas de transmisión por correa, que conectan un motor ubicado en la parte inferior externa con la plataforma giratoria que se encuentra por encima. Esta separación entre el motor y la zona móvil permite minimizar la masa en movimiento, facilitando un mejor rendimiento dinámico.

Además, en la parte superior de la base se encuentra un orificio pasante cuidadosamente diseñado, por el cual se transmite el movimiento rotacional mediante un eje acoplado al motor y conectado a una polea que acciona la tapa giratoria. Esta tapa descansa sobre un rodamiento de gran tamaño, lo que garantiza un giro suave, preciso y con baja fricción, permitiendo que el brazo pueda rotar sobre el eje Z de forma eficiente

Tapa giratoria

La tapa giratoria es el elemento clave que permite la rotación horizontal del brazo robótico sobre su eje vertical (eje Z). Esta pieza se sitúa justo encima de la base cilíndrica fija y constituye la plataforma sobre la que se apoya y ensambla el resto de la estructura del brazo.

Su diseño ha sido totalmente personalizado en Fusion 360 para adaptarse a los componentes internos del sistema de transmisión, al rodamiento inferior y a las piezas estructurales del brazo. Está compuesta por una placa circular rígida, que encaja perfectamente sobre el rodamiento, lo que le permite un movimiento giratorio fluido sin generar oscilaciones ni desalineamientos.

Esta tapa se acciona mediante una correa dentada, conectada a una polea fijada en su parte inferior, que a su vez está impulsada por un motor montado externamente. La elección de una transmisión por correa permite mantener el motor lejos del eje rotatorio, reduciendo el peso en la parte móvil y facilitando el mantenimiento. Además, este sistema proporciona una relación de transmisión precisa y silenciosa, clave para el control fino de la orientación del brazo.

Otro detalle importante es que la tapa está diseñada con anclajes integrados para los eslabones del brazo y para el paso de cables desde la parte fija hasta la parte móvil, evitando que se enreden o deterioren durante el giro.

Primer eslabón del brazo

El primer eslabón del brazo está construido a partir de un tubo de aluminio de sección cuadrada, una elección que ofrece una buena combinación de ligereza y resistencia mecánica. Este tubo se ha perforado y cortado estratégicamente para permitir la instalación de componentes de transmisión mecánica, como poleas y rodamientos, que permiten el movimiento de este eslabón y su articulación con el siguiente segmento del brazo.

Los cortes y perforaciones realizadas en el tubo están cuidadosamente distribuidos para optimizar el paso de las correas de transmisión, sin comprometer la rigidez del eslabón. De esta manera, se asegura que la estructura sea lo suficientemente fuerte como para soportar las cargas mecánicas generadas durante el movimiento del brazo, mientras que al mismo tiempo se facilita la integración de los sistemas de transmisión, permitiendo un funcionamiento suave y controlado.

Este eslabón está conectado a las piezas impresas en 3D, que han sido diseñadas específicamente para facilitar las articulaciones del brazo. Las piezas impresas sirven como soportes para los rodamientos que permiten la rotación entre eslabones, y también se encargan de alojar las poleas y guiar las correas de transmisión que gestionan los movimientos. Al estar diseñadas a medida en Fusion 360, las piezas impresas garantizan un ajuste perfecto con el tubo de aluminio, lo que optimiza tanto la precisión del movimiento como la durabilidad del sistema.

Este diseño modular, que combina el uso de aluminio para los eslabones y de piezas impresas en 3D para las articulaciones, no solo permite una estructura resistente y ligera, sino también flexible y fácil de modificar en caso de ser necesario.

Segundo eslabón del brazo (antebrazo) y barras secundarias

El segundo eslabón del brazo sigue el mismo diseño que el primero, utilizando tubo de aluminio perforado para integrar las correas de transmisión y los rodamientos. Se conecta al primer eslabón mediante piezas impresas en 3D, que permiten un movimiento articulado y aseguran la correcta alineación de las correas. Las barras secundarias refuerzan la estructura, aportando estabilidad y resistencia a la flexión, y contribuyen a la rigidez general del brazo, mejorando su desempeño durante el funcionamiento.

Actuador final

El actuador final es una pala de excavadora convencional, diseñada específicamente para facilitar la recogida de arena mojada. Su diseño incluye una rampa en la parte inferior para asegurar que la arena se desplace de manera eficiente hacia el interior de la pala, mejorando la capacidad de carga y optimizando el proceso de recolección. La pala está directamente conectada al motor, lo que permite un control preciso del movimiento de apertura y cierre, adaptándose a las diferentes cantidades de arena a recoger.

Este diseño garantiza un alto rendimiento en la tarea de traslado de arena, asegurando que el robot pueda operar con eficacia incluso en condiciones de arena mojada.

Dimensiones y capacidades

El brazo robótico ha sido diseñado con dimensiones que permiten una operación eficiente en un área de trabajo de al menos 20x20 cm, que es el tamaño mínimo requerido para recoger la arena de una superficie plana. Las dimensiones generales del brazo son compactas, lo que permite un manejo adecuado en espacios reducidos y mejora la maniobrabilidad.

La capacidad de carga de la pala ha sido ajustada para permitir que el robot pueda recoger una cantidad significativa de arena mojada, sin comprometer la estabilidad ni el rendimiento del sistema. La pala y los motores están diseñados para transportar entre 1 y 1,5 kg de arena, lo que asegura que el robot pueda completar tareas de recolección en un tiempo razonable.

4.2 Primeras versiones del diseño 3D

Para tratar de controlar de una manera más directa el diseño del robot, decidimos abordar el problema desde 0 apostando por diseñar en 3D para después imprimirlo. De esta manera nos aseguramos que el robot cumpla con las especificaciones que necesitamos y nos permite más flexibilidad a la hora de elegir los componentes. Esto era un elemento crucial para poder introducir los componentes electrónicos en el modelo ya que elegimos motores con un alto par, con unas dimensiones más grandes de lo

habitual. Por ello, necesitábamos adaptar el robot para poder montar los motores y los reductores sin comprometer la estabilidad del diseño.

El diseño de nuestro robot cuenta con 4 partes diferenciadas de las que abordaremos tanto el proceso creativo como la progresión en las versiones de diseño. Hemos creído pertinente incluir parte del desarrollo además de las versiones finales para entender el proceso.

Lo primero que el equipo de diseño decidió diseñar, fue un modelo de los componentes ajenos al diseño. Esto permitió familiarizarnos con el programa al no tener que preocuparnos de pensar en el diseño de las piezas y simplemente copiamos los componentes utilizando las medidas proporcionadas por el fabricante.

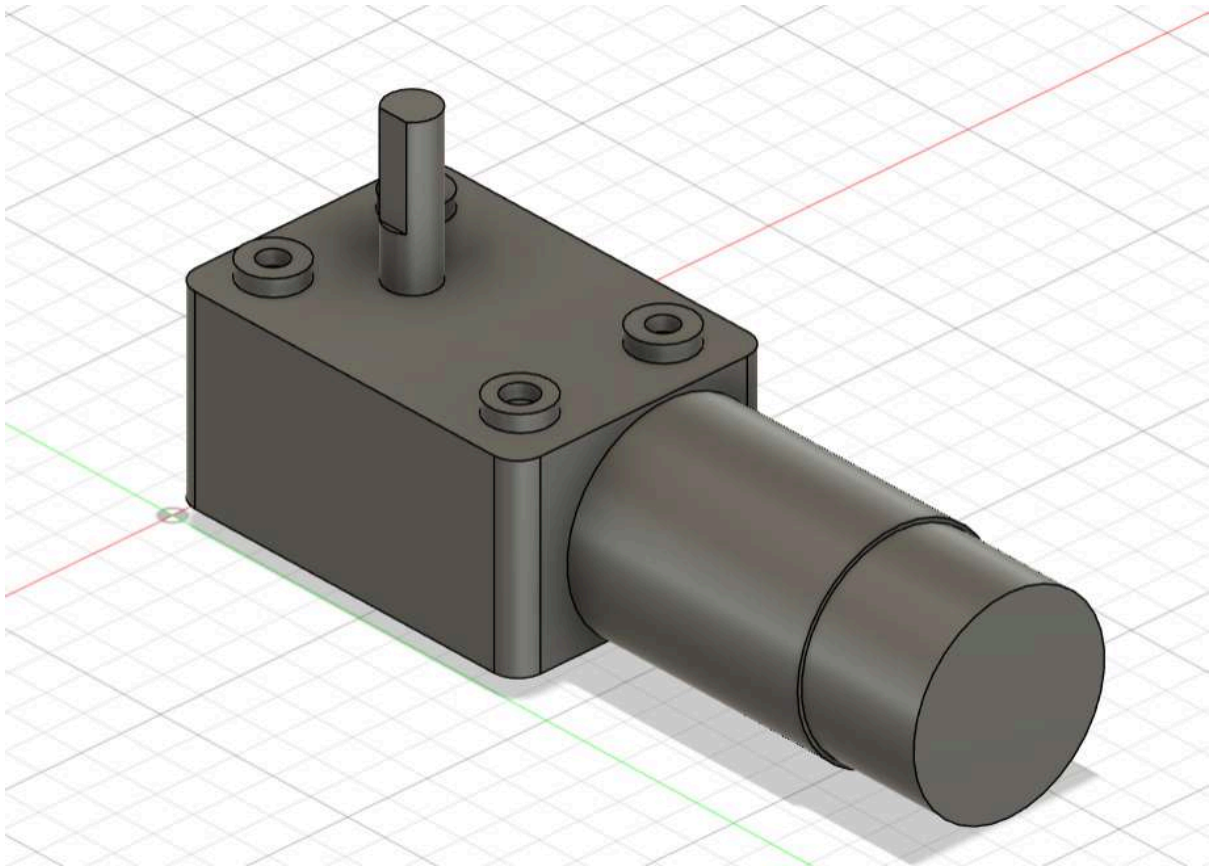


Figura 1: Motor con codificador efecto hall y eje helicoidal (Versión Final)

Esto nos sirvió como introducción a el programa antes de ponernos a pensar en soluciones para abordar las diferentes articulaciones y el modo de conexión con la base

Después de una reunión del equipo de diseño, sacamos el primer diseño con el que ir iterando. En esta etapa solo nos centramos en las articulaciones de los brazos, articulaciones solidarias en los brazos del robot sin usar todavía rodamientos. Estos pensábamos que irían conectados directamente a los motores aunque rápidamente nos dimos cuenta que no era viable:

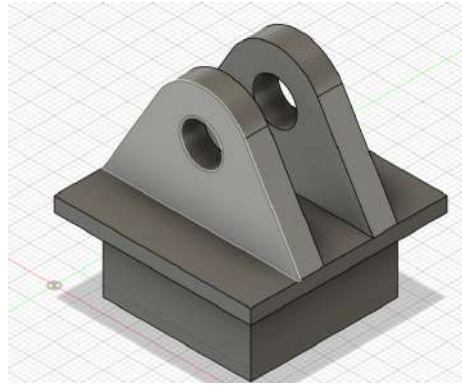


Figura 2: Pieza de diseño articulación 1

Esta es la primera versión de la articulación no solidaria entre los brazos. Ya tiene el contorno final de 40 mm de lado aunque las dimensiones no aceptan muchos más elementos en el medio.

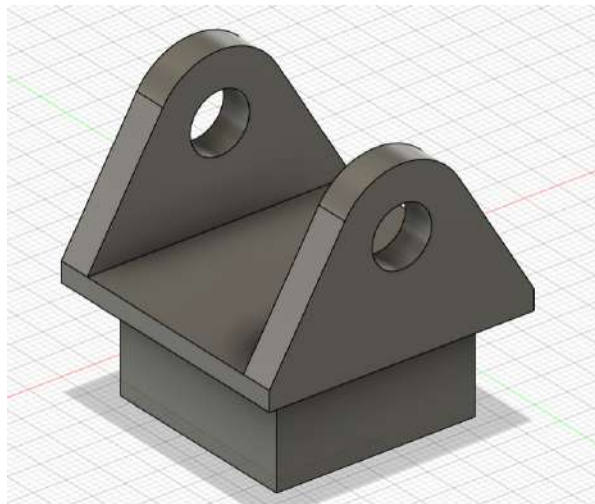


Figura 3: Pieza de diseño articulación 2

Simplemente pensamos en ajustar ambas aplicaciones con un eje pero al ser ambos solidarios con el giro desechamos la idea.

Una vez familiarizados con el entorno de Fusión, usamos el modelo de una pala real para sacar el primer diseño para la pala:

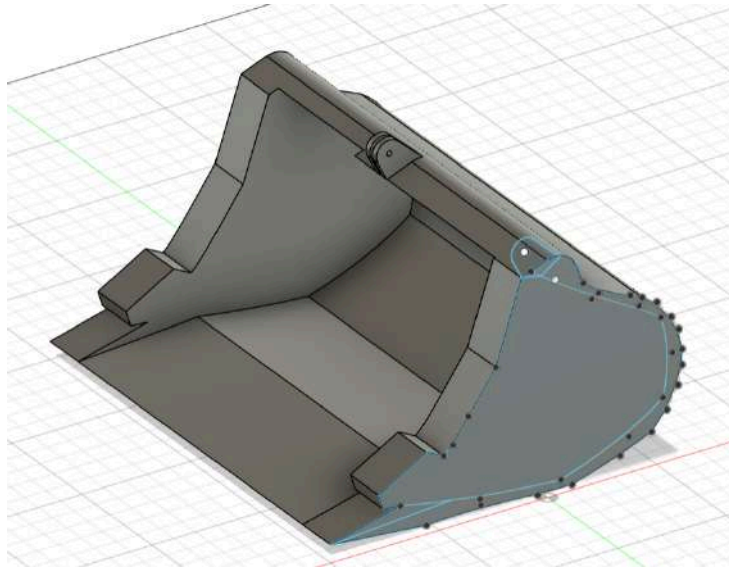


Figura 4: Primer modelo de la pala

Tras este primer modelo, cambiamos varias cosas como, las dimensiones para que se ajustaran a las del robot, la inclinación del suelo de recogida para que fuera más fácil recoger la arena, alargar la conexión con el eje de giro para dar más movilidad a la pala, cambiar el diámetro del agujero del eje para que se ajustara exacto con el eje del motor, hacer las paredes de la pala más fina para hacerla más ligera y gastar menos filamento de impresión.

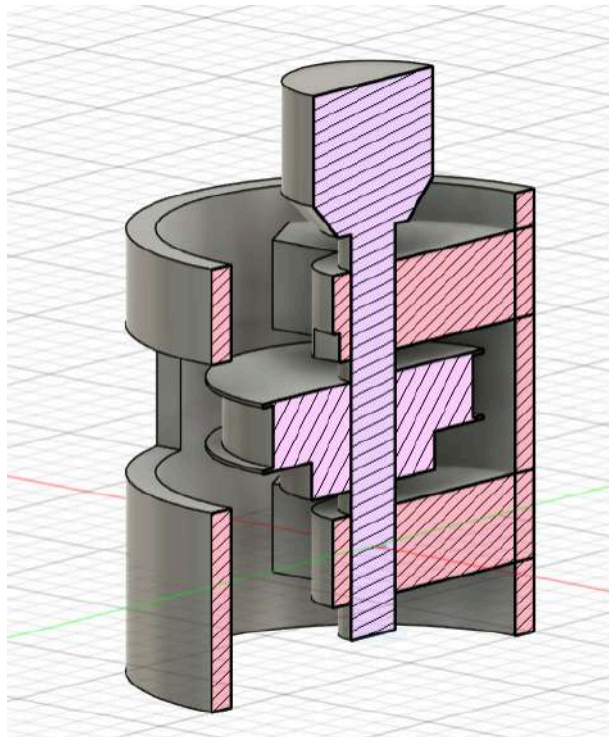


Figura 5: Enganche brazo con el suelo, primer modelo

Este es el primer modelo de la conexión con el suelo, en el se ve el eje que hace girar al robot, con los dos soportes para sujetar los rodamientos (sin el rodamiento todavía ya

que desconocíamos cuales íbamos a utilizar.), el engranaje grande de la reductora y un agujero por el que pasará la banda de la reductora. Todavía no tenía un soporte para el motor y para el engranaje pequeño de la reductora.

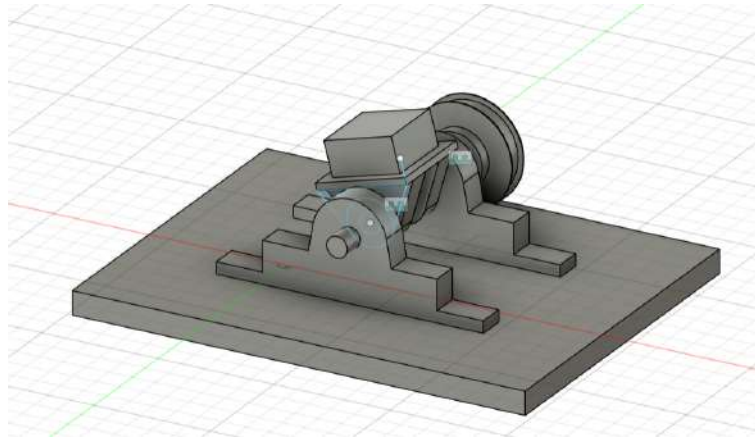


Figura 6: Plataforma base

Por último esta es la primera versión de la pieza a la que haría rotar la pieza anterior, la que le daría la rotación a todo el brazo, ya cuenta con las articulaciones solidaria del brazo y no solidaría de la base (también sin espacio para los rodamientos), los engranajes y la plataforma de la base. Tras esta versión, nos dimos cuenta rápido de que era innecesario tanto material y que podíamos hacer el modelo mucho más eficiente.

4.3 Modelo final

Esta primera versión nos permitió tener una idea, aunque aun fuera un poco abstracta de lo que iba a resultar el diseño final del brazo. Después de este modelo, seguimos iterando para finalmente conseguir una solución que nos satisfaga. A continuación, el diseño de todas las piezas que imprimimos para el modelo final del brazo robótico.

4.3.1 SWIVEL

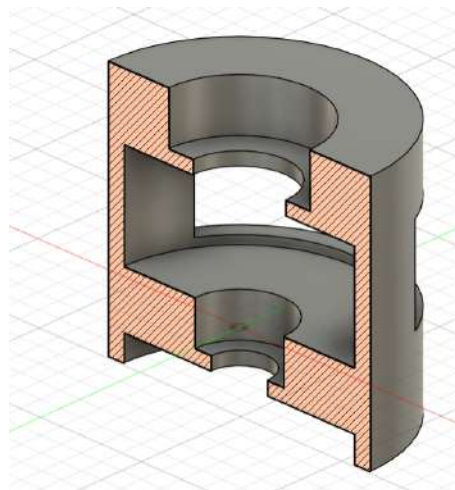


Figura 7: Estructura de la base

La primera de las piezas es la encargada de sujetar el eje del motor adaptado, los engranajes y los rodamientos (608zz abajo y 6202z el de arriba) , también dispone de un agujero para dejar pasar la banda de la reductora. Tras muchas iteraciones, llegamos a una caja con un diámetro de 65mm, y una ventana de 22mm de alto.

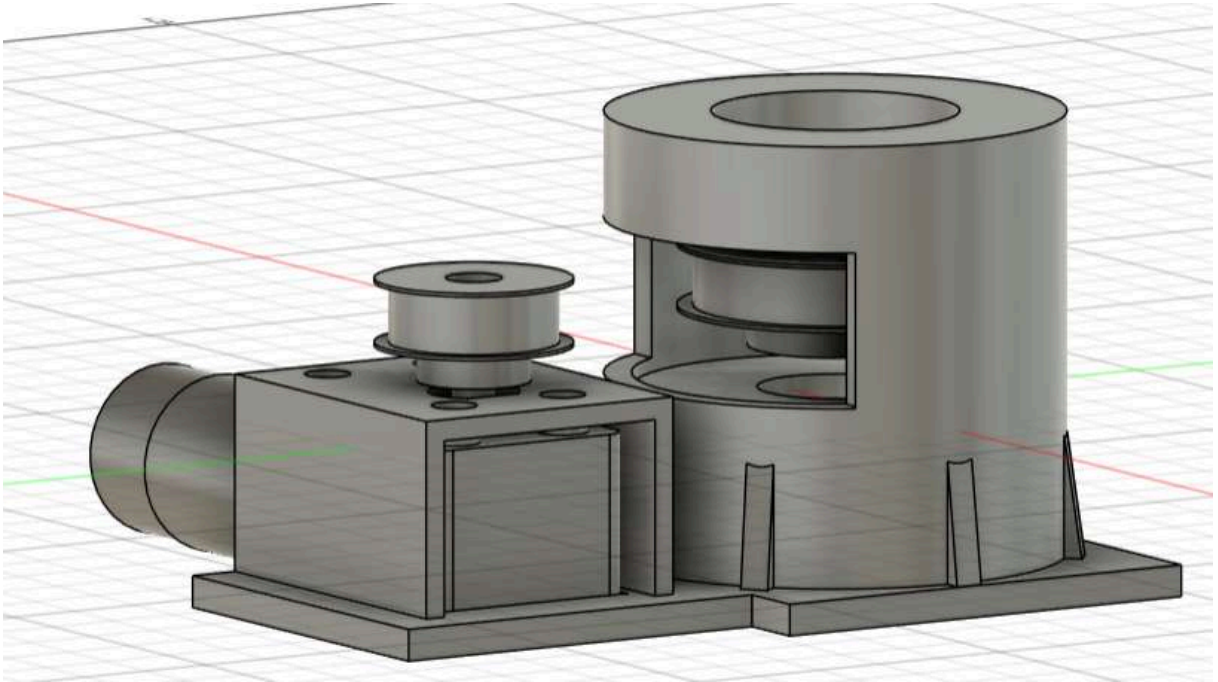


Figura 8: Caja + soporte para el motor + unión al suelo (Versión final)

Necesitábamos una pieza para sostener el motor, con la idea de que ambos engranajes quedarán a la misma altura. Además, añadimos una plataforma para darle más estabilidad a la unión con el suelo, de esta manera podíamos clavar el sistema al tablón de madera y así no tener que depender de la fuerza del pegamento. El motor estará sujeto al puente mediante tornillos, en los agujeros adaptados para ellos en la parte superior y añadimos pequeños nervios a la caja para darle más resistencia frente a los esfuerzos normales del eje.

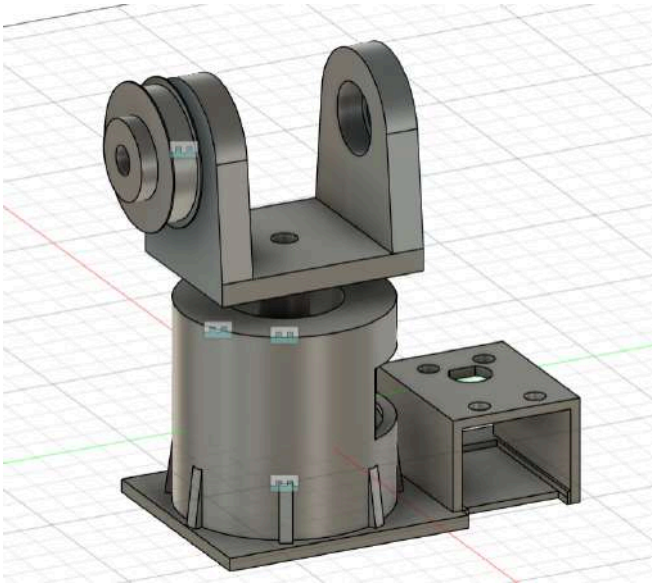


Figura 9: Estructura base completa

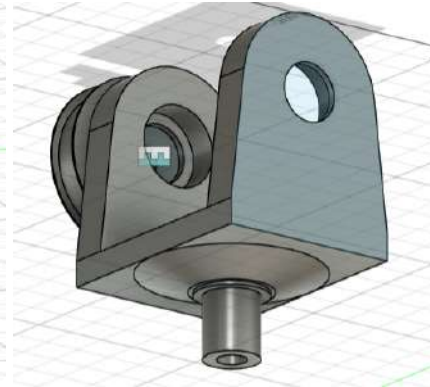


Figura 10: Conexión Base-Eslabón

La última de las piezas que debíamos imprimir en esta parte del robot era la pieza que conectara la articulación de rotación del robot, con la articulación no solidaria del eje. Finalmente nos decantamos por imprimir solo la parte superior del eje de rotación y añadir un pasante para que se introdujera en su interior un eje de acero inoxidable de 8mm de diámetro. Con esta solución conseguimos la resistencia del acero, unido a la flexibilidad y el buen encaje de las piezas.

Tal y como lo diseñamos, el eje sería de 8mm para pasar por el orificio del rodamiento 6202 zz y al salvar este agujero, ensancharlo con la pieza para asegurar que hubiera contacto con la parte solidaria del rodamiento para conseguir un movimiento armónico de rotación de toda el conjunto. También, en la parte superior la pieza tiene huecos para encajar con los dos rodamientos 608 zz necesarios para hacer el movimiento no solidario y evitar una posible rotura. Con esta pieza, concluimos con la parte del robot encargada del “swivel”.

4.3.2 Pitch

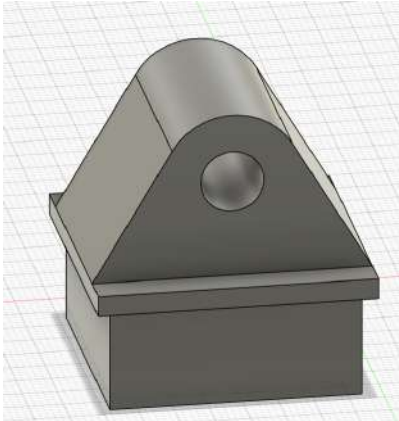


Figura 11: Enganche eslabón

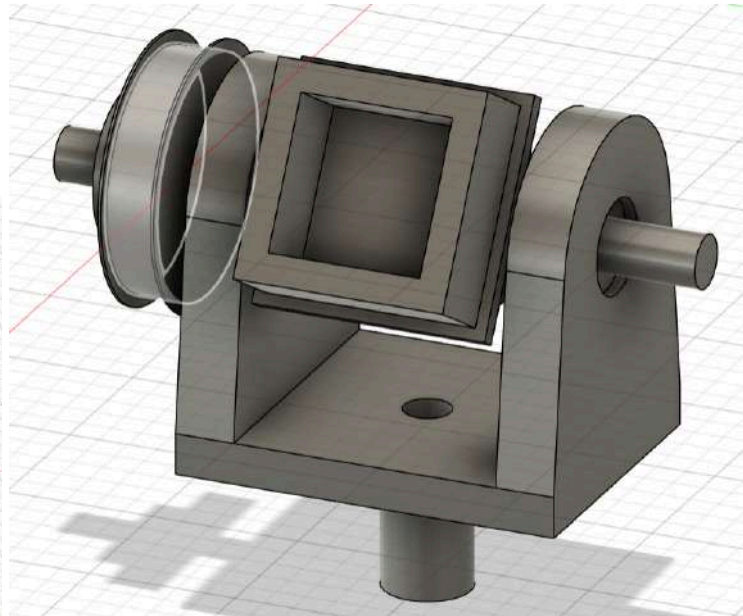


Figura 12: Montaje conexión eslabón

La siguiente parte del brazo de la que vamos a hablar es la encargada del movimiento de pitch. El mayor desafío de este movimiento era la posición del motor encargado de realizar el movimiento. En un primer momento pensamos en, al movimiento ser fijo siempre en un punto fijo dejarlo fuera pero de esta manera renunciábamos la rotación completa del robot por lo cual lo descartamos. También pensamos en prescindir de la reductora en el robot pero esta era la articulación que iba a necesitar más par de las 3 y si queríamos tener un robot de un tamaño decente íbamos a necesitar incluirla.

Finalmente diseñamos una manera para incluir el motor y la reductora dentro del propio eslabón del brazo para que lo acompañara durante su movimiento. Pondremos el engranaje grande de la transmisión en el eje de giro y haremos un agujero en la pared del brazo para dejar salir tanto el motor como el engranaje pequeño. Además tendremos que atornillar el motor a la pared del brazo y hacer un orificio para introducir los cables de realimentación y control.



Figura 13: Esquema del montaje del motor y la reductora dentro del brazo del robot.

4.3.3 Elbow

El siguiente grupo de piezas son la unión entre los dos brazos, los encargados de hacer el movimiento de elbow. El desafío de esta articulación fue la longitud de las piezas. Cómo de largas debemos hacer las piezas para permitirnos tener el mayor giro posible. Al ser una pieza sencilla pero con tantos aspectos pequeños, este fue la articulación con más iteraciones llegando a tener hasta 19 versiones distintas hasta dar con la versión definitiva.

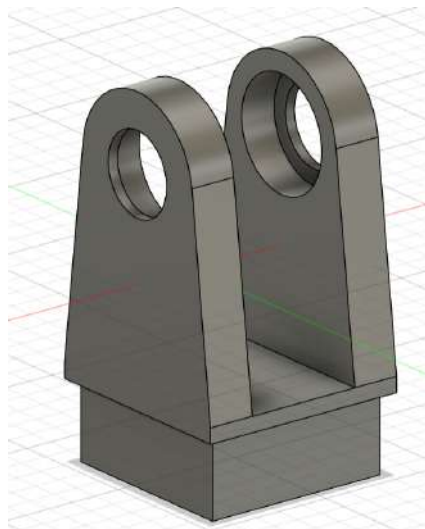


Figura 14: Articulación no solidaria unión dos eslabones(Versión definitiva)

En primer lugar está la articulación que se encuentra al final del primer brazo. Esta articulación se pegara a este y no será solidaria con el movimiento de elbow. Tiene

espacio para albergar los rodamientos 608 zz y tiene una altura hasta el eje de 43 mm, que consideramos que era lo óptimo para ganar movilidad sin comprometer estabilidad.

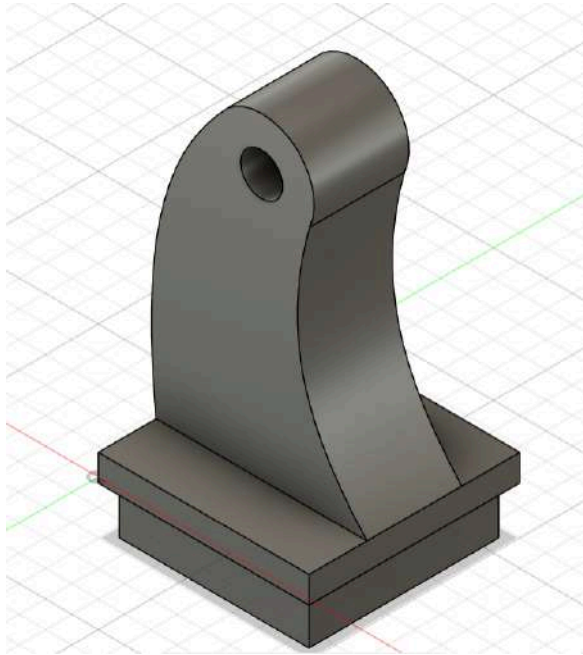


Figura 15: Articulación solidaria unión dos eslabones (versión final)

Esta es la articulación solidaria con el eje del giro, tiene una altura de 50mm desde el eje y cuenta con una ligera curvatura para admitir todavía más giro. Pese a que esta articulación es muy larga, el hecho de que sea tan gruesa y robusta hace que no nos tengamos que preocupar en exceso por su resistencia a esfuerzos.

Estas dos articulaciones estarían conectadas de una manera similar a la articulación inferior, con el motor del mismo modo atornillado en las paredes del eslabón.

4.3.4 Pala

Por último tenemos el elemento fundamental del diseño, la pala. Para esta parte la mayor preocupación eran las dimensiones. Hicimos estimaciones teniendo en cuenta el par de los motores para ver cuanta arena íbamos a ser capaces de rescatar. Además tuvimos que ver cómo conectábamos la pala. Al no contar con una reductora, no disponíamos de un eje circular para conectarlo por lo que tuvimos que adaptar el diseño al eje del motor para que coincidiera perfecto y no hubiera juegos innecesarios.

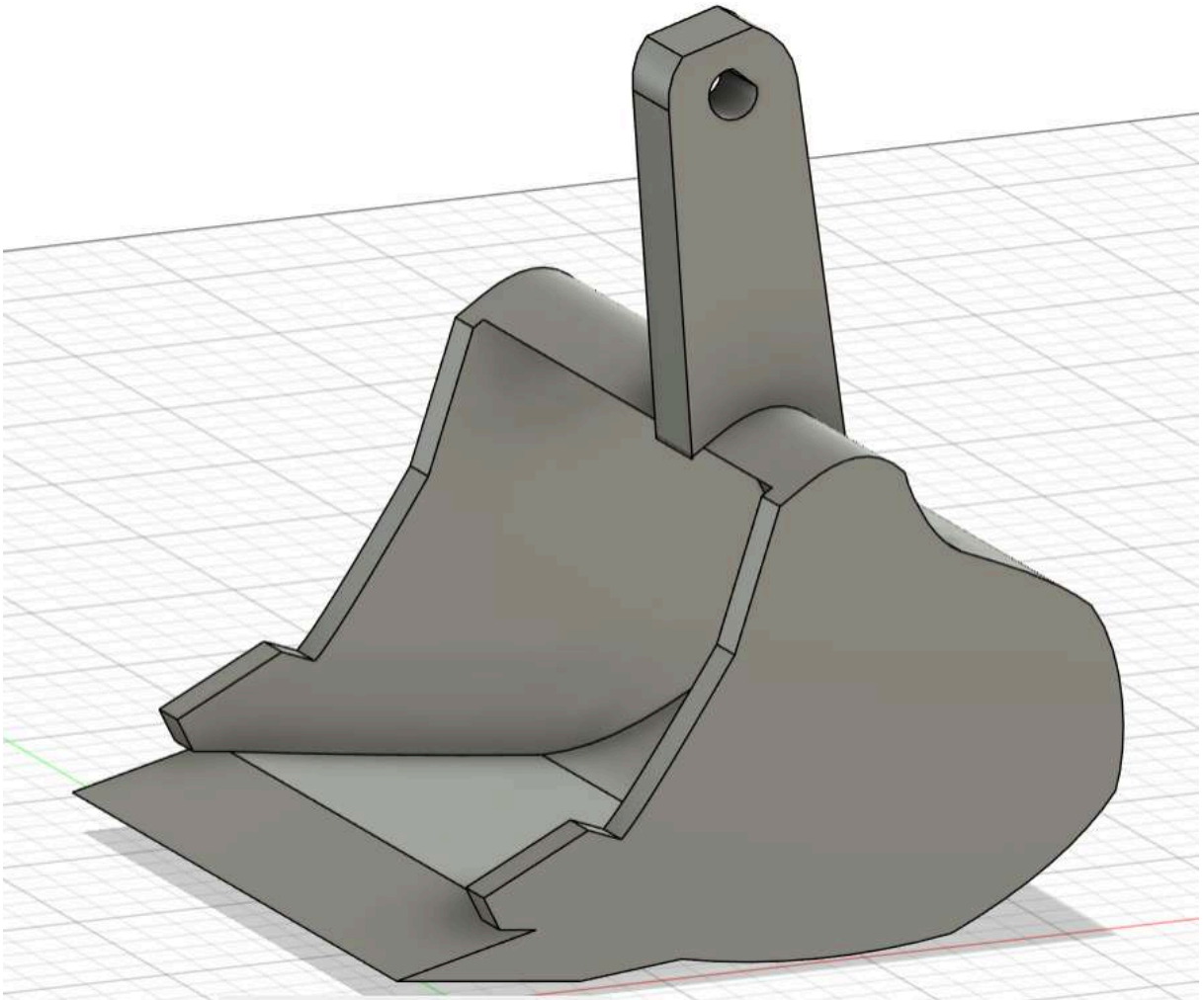


Figura 16: Pala (Versión final)

El diseño final cuenta con hendiduras en puntos estratégicos para que la arena no se salga una vez recogida, paredes delgadas que hacen a la herramienta más ligera, una extremidad larga para asegurar que la pala no choca con el brazo, movida hacia un lateral para mantener el centro de gravedad estable. Además incluimos una llanura en la parte superior de la extremidad para admitir clavar un tornillo al eje y asegurar que no se despegue en ningún momento.

Con esta pieza terminan todas las piezas que nos hemos encargado de diseñar para unir los eslabones y darle al brazo los grados de libertad que deseábamos al principio del proceso. Sin duda, hemos visto mejoría en nuestra destreza con la herramienta de diseño y hemos querido incluir el proceso de evolución del diseño para dar más perspectiva a nuestro brazo robótico.

A continuación la esquematización del brazo completo:

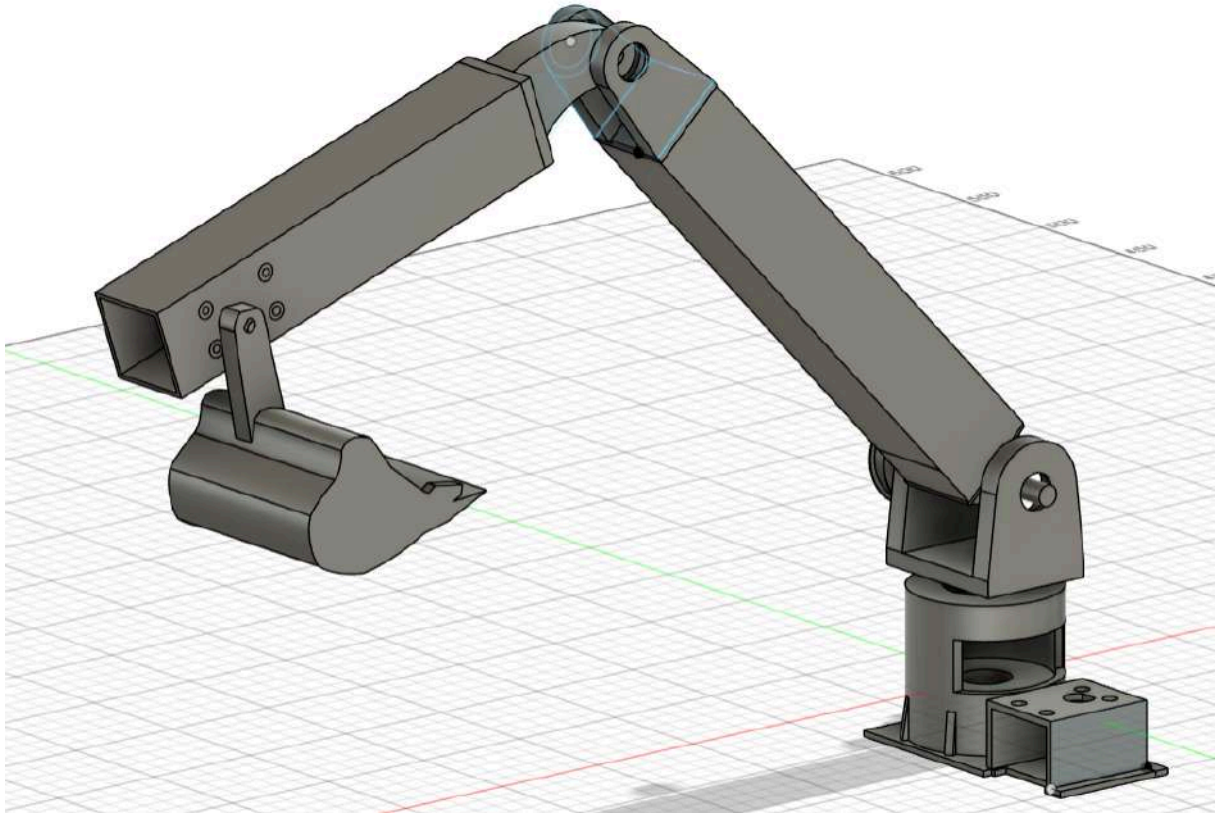


Figura 17: Esquematación brazo entero (Versión final)

4.4 Cinemática

La cinemática del brazo robótico se ha desarrollado para garantizar un movimiento suave y controlado. Utilizando una cadena cinemática cerrada, los movimientos de los eslabones son gestionados por correas de transmisión que conectan los motores con los rodamientos en las articulaciones del brazo. El sistema de transmisión se controla mediante una serie de sensores y actuadores, lo que permite un ajuste preciso de la posición del brazo en tiempo real.

Cinemática Directa

Para el análisis cinemático del brazo robótico diseñado, se ha partido de un modelo tridimensional simplificado compuesto por tres grados de libertad principales: rotación de la base, articulación del primer eslabón (hombro) y articulación del segundo eslabón (antebrazo). La base giratoria, aunque funcionalmente relevante, se ha considerado como un grado adicional aislado respecto al modelo cinemático directo, ya que su movimiento es ortogonal al plano principal de trabajo del brazo.

El sistema de articulaciones es totalmente rotacional. Cada articulación conecta un eslabón con el siguiente mediante una pieza impresa en 3D que integra rodamientos y poleas dentadas, facilitando un movimiento preciso gracias al uso de motores alojados internamente en los eslabones. Esta arquitectura minimiza la masa del extremo operativo y mejora el reparto de cargas, siguiendo un enfoque similar al de los brazos industriales articulados tradicionales.

El modelo de cinemática directa se centra en obtener la posición del extremo del efector (la pala de recogida de arena) en el espacio cartesiano tridimensional (x,y,z) , a partir de los ángulos de las articulaciones. Para simplificar la expresión matemática, se define un sistema de coordenadas base fijo al suelo, con el eje Z apuntando hacia arriba, el eje X hacia el frente del robot y el eje Y hacia la izquierda desde la perspectiva del operador.

Dado que se trata de un brazo con dos eslabones principales y tres articulaciones rotacionales (incluyendo la base), el posicionamiento espacial se puede describir a partir de la cadena cinemática de tipo R-R-R (rotacional-rotacional-rotacional), siendo el modelo equivalente al de un manipulador tipo SCARA modificado para operar en 3D.

Los parámetros utilizados para definir la cinemática se moldearán posteriormente mediante la convención de Denavit-Hartenberg, una metodología estándar para representar geometrías robóticas. La aplicación de dicha convención permitirá definir los desplazamientos y rotaciones relativos entre eslabones, así como establecer las matrices de transformación homogénea que relacionan el sistema de coordenadas de la base con el del efector final.

Esta modelización no solo es esencial para el cálculo de trayectorias y posicionamiento del brazo, sino que resulta bastante útil a la hora de realizar la cinemática inversa que es la parte más importante a implementar al brazo ya que todo su movimiento se basa en ella.

Cinemática Inversa

La cinemática inversa se utiliza para calcular los ángulos de las articulaciones del brazo necesarios para alcanzar una posición deseada en el espacio. El sistema de control resuelve las ecuaciones de la cinemática inversa en tiempo real, ajustando los movimientos de las articulaciones para que la pala se desplace hacia la ubicación exacta donde debe recoger la arena.

Para la resolución de la cinemática inversa de nuestro brazo robótico, hemos optado por emplear un modelo trigonométrico en lugar de recurrir a métodos más algebraicos o matriciales, como el uso intensivo de transformaciones homogéneas y el álgebra lineal. Esta elección responde principalmente a criterios de claridad, adecuación al problema y optimización del tiempo de desarrollo, tal como se detalla a continuación:

1. Simplicidad del sistema.

Nuestro brazo robótico, a pesar de ser funcional y robusto, cuenta con solo tres grados de libertad y una estructura planar relativamente sencilla (cuando se proyecta en el plano vertical). Esto permite que el análisis trigonométrico con triángulos y funciones trigonométricas básicas (seno, coseno, tangente) sea no sólo válido, sino más directo y comprensible.

2. Mejor visualización geométrica.

El uso de triángulos y relaciones angulares facilita enormemente la visualización del sistema, tanto para comprender el funcionamiento del brazo como para detectar errores en los cálculos o en el diseño. Esta ventaja es especialmente útil cuando se trabaja con herramientas como Fusion 360, donde el diseño 3D se apoya en planos y cotas que reflejan claramente estos elementos geométricos.

3. Evita complejidad innecesaria.

Aunque el uso de matrices de rotación y transformaciones homogéneas es muy potente y versátil, su complejidad no está justificada en un sistema tan reducido en grados de libertad como el nuestro. Además, para estudiantes e ingenieros en fase de desarrollo, el enfoque trigonométrico permite una mayor comprensión intuitiva del comportamiento del robot.

4. Facilidad de implementación y depuración.

El modelo trigonométrico se traduce más fácilmente en un conjunto de fórmulas secuenciales que pueden implementarse de forma sencilla en cualquier lenguaje de programación o incluso en una hoja de cálculo. Además, al estar basado en pasos lógicos visibles, es más fácil de depurar si algo falla.

Nuestro brazo tiene:

Una base rotatoria (θ_1): gira en horizontal, en el plano XY.

Un plano vertical articulado con dos eslabones:

-Primer eslabón de longitud L_1 con ángulo de elevación θ_2 .

-Segundo eslabón de longitud L_2 con ángulo relativo θ_3 .

Se busca alcanzar un punto deseado en el espacio (x, y, z).

Paso 1: θ_1 , ángulo de base (horizontal)

La proyección del punto (x, y, z) sobre el plano XY nos da el ángulo con respecto al eje X. Eso es simplemente:

$$\theta_1 = \text{atan2}(y, x)$$

Se usa atan2 para tener en cuenta el signo de x e y , y obtener el ángulo correcto en el cuadrante adecuado.

Paso 2: Reducción al plano vertical

Una vez determinada la orientación horizontal con θ_1 , nos interesa el plano vertical en el que se encuentra el brazo. Este plano contiene los ejes Z (vertical) y R, donde:

$$R = \sqrt{x^2 + y^2} \rightarrow \text{distancia horizontal desde la base al punto.}$$

Entonces el problema se reduce a un brazo plano en el plano RZ.

Paso 3: Cinemática inversa planar

Dado un punto (R, z) y longitudes L_1 y L_2 , queremos obtener los ángulos θ_2 y θ_3 .

La fórmula clave proviene del Teorema del Coseno aplicado al triángulo formado por L_1 , L_2 y la línea que conecta el origen al punto (R, z):

$$D = (R^2 + z^2 - L_1^2 - L_2^2) / (2 L_1 L_2)$$

Entonces:

$$\theta_3 = \text{atan2}(\pm\sqrt{1 - D^2}, D)$$

Usamos $\pm$ para reflejar que hay dos soluciones posibles:

codo hacia abajo

codo hacia arriba

Paso 4: Calcular θ_2

Para encontrar θ_2 , usamos:

$\beta = \text{atan2}(z, R) \rightarrow$ ángulo entre la línea base-punto y el eje horizontal

$\alpha = \text{atan2}(L_2 \cdot \sin(\theta_3), L_1 + L_2 \cdot \cos(\theta_3)) \rightarrow$ ángulo entre L_1 y la línea base-punto

Entonces:

$$\theta_2 = \beta - \alpha$$

Todas las fórmulas salen de descomponer el problema en dos partes:

Rotación en XY $\rightarrow \theta_1 = \text{atan2}(y, x)$

Análisis trigonométrico en el plano RZ para θ_2 y θ_3 , usando el teorema del coseno y relaciones trigonométricas entre triángulos.

-Finalmente tendríamos los siguientes datos y fórmulas que será implementado en la programación.

Datos del robot

L_1 = longitud del primer eslabón

L_2 = longitud del segundo eslabón

θ_1 = rotación en plano horizontal (alrededor del eje Z)

θ_2 = ángulo de elevación del primer eslabón respecto a la horizontal

θ_3 = ángulo entre el primer y segundo eslabón (ángulo relativo)

Punto deseado (x, y, z): posición objetivo del extremo del brazo.

Cinemática inversa

(dado x, y, z $\rightarrow$ obtener θ_1 , θ_2 , θ_3)

1. Cálculo auxiliar:

$$R = \sqrt{x^2 + y^2}$$

2. $\theta_1 = \text{atan2}(y, x)$

3. $D = (R^2 + z^2 - L_1^2 - L_2^2) / (2 \cdot L_1 \cdot L_2)$

4. Posibles soluciones para θ_3 :

$$\theta_3 = \text{atan2}(\pm\sqrt{1 - D^2}, D)$$

(se puede elegir signo positivo para codo hacia arriba o negativo para codo hacia abajo)

5. Luego:

$$\beta = \text{atan2}(z, R)$$

$$\alpha = \text{atan2}(L_2 \cdot \sin(\theta_3), L_1 + L_2 \cdot \cos(\theta_3))$$

$$\theta_2 = \beta - \alpha$$

Recálculo de las posiciones

El sistema está diseñado para recalcular las posiciones de los eslabones de manera continua durante la ejecución de la tarea. Este recálculo se realiza en función de la información de los sensores y el feedback de los motores, asegurando que el brazo se mantenga en la posición correcta a medida que avanza con la tarea de recolección y transporte de la arena.

5. DISEÑO ELECTRÓNICO

5.1 Componentes electrónicos

5.1.1 Motor corriente continua con encoder

Un motor de corriente continua (DC) es un actuador eléctrico que convierte energía eléctrica en movimiento rotativo continuo, cuya dirección y velocidad pueden controlarse fácilmente.

Para este proyecto se ha optado por el uso de motores de corriente continua con encoder y reductora interna, debido a su capacidad para proporcionar un par elevado .

Se han utilizado dos modelos distintos según la función de cada articulación:

- Motores de 10 rpm, en la base y pala, donde conviene tener un desplazamiento angular relativamente rápido y no requiere tanto par.
- Motores de 5 rpm, donde se necesita mayor par para elevar el brazo.



Figura 18: Motor corriente continua

Todos los motores incorporan un encoder magnético basado en sensores de efecto Hall, que genera dos señales digitales (A y B) en cuadratura, es decir, desfasadas 90° entre sí. Este tipo de señal permite no solo contar los pasos del eje, sino también determinar el sentido de giro:

- Si la señal A adelanta a la B → giro horario (CW).
- Si la señal B adelanta a la A → giro antihorario (CCW).

Gracias al encoder, podemos conocer el desplazamiento angular del eje del motor.



Figura 19: Encoder y conexiones

Además de la reductora interna del motor, en los ejes de la base, hombro y codo se ha añadido una transmisión mecánica por engranajes para adaptar aún más el par y la velocidad en función del requerimiento de cada articulación.

Especificaciones:

Base y pala: Motor DC con encoder – 10 rpm

Especificación eléctrica:

- Tensión nominal: 12 V
- Corriente en vacío: ~100 mA
- Corriente con carga: hasta 1 A
- Velocidad sin carga: 10 rpm
- Reductora interna 1:600
- Engranajes 40:60 (sólo en la base)
- Encoder: 11 pasos/vuelta

Especificación de control:

- Control mediante puente H (L298N)
- Dirección de giro controlada por IN1/IN2
- Velocidad: regulada mediante señal PWM

Encoder:

- Alimentación: 5 V
- Salida: digital (dos señales desfasadas 90°)
- Detección de sentido

Hombro y codo: Motor DC con encoder – 5 rpm

Especificación eléctrica:

- Tensión nominal: 12 V
- Corriente en vacío: ~100 mA
- Corriente con carga: hasta 1.2 A
- Velocidad sin carga: 5 rpm
- Reductora interna 1:900
- Engranajes: 16:60
- Encoder: 11 pasos/vuelta

Especificación de control:

- Control mediante puente H (L298N)
- Dirección de giro: controlada por IN1/IN2
- Velocidad: regulada mediante señal PWM

Encoder:

- Alimentación: 5 V
- Salida: digital (dos señales desfasadas 90°)
- Detección de sentido

Pares de los motores

Motor de la base (10 rpm, engranajes): $1.875 \text{ N} \cdot \text{m}$

Motor del hombro y codo (5 rpm, engranajes): $8,5 \text{ N} \cdot \text{m}$

Motor de la pala (10 rpm, sin engranajes): $1.25 \text{ N} \cdot \text{m}$

Dimensiones del motor de corriente continua.

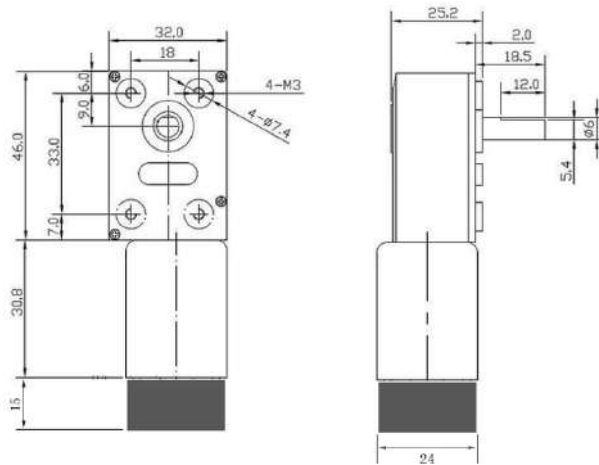


Figura 20: Medidas motor CC

5.1.2 L298N - Puente H

El puente H permite un control sencillo tanto del sentido de giro del motor como de la velocidad.

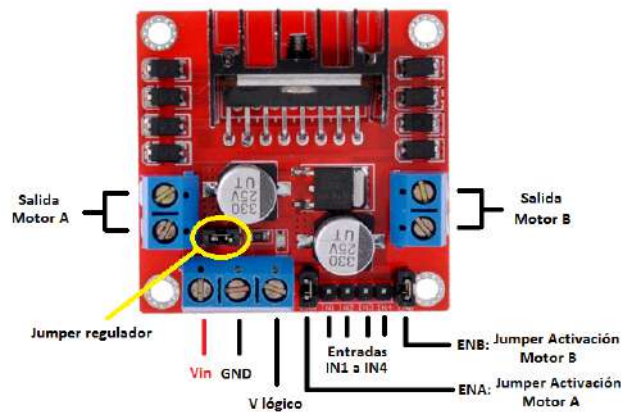


Figura 21: Esquema de pines puente H

Con este componente podemos controlar el sentido de la corriente eléctrica en uno u otro sentido mediante el uso de cuatro transistores.

La disposición de estos transistores hace posible que, al querer que el motor gire en un sentido, se activen dos de ellos; y al querer que gire en el sentido contrario, se activen los otros dos.

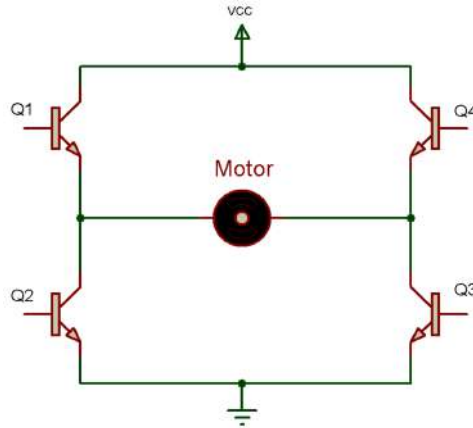


Figura 22: Esquema electrónico de puente H con un motor CC

Esta configuración recibe el nombre de "puente H" debido a su disposición en forma de letra H, con el motor conectado en el centro.

En el caso de nuestro motor de corriente continua, si activamos los transistores Q1 y Q3, la corriente fluye en un sentido a través del motor, en cambio, si activamos Q2 y Q4, la corriente circula en sentido contrario, invirtiendo así la dirección del giro.

Existen diferentes módulos que implementan este circuito, en nuestro caso, emplearemos el L298N, un integrado que incorpora dos puentes H, lo que nos permite controlar simultáneamente dos motores de corriente continua.

El esquema de conexión es el siguiente:

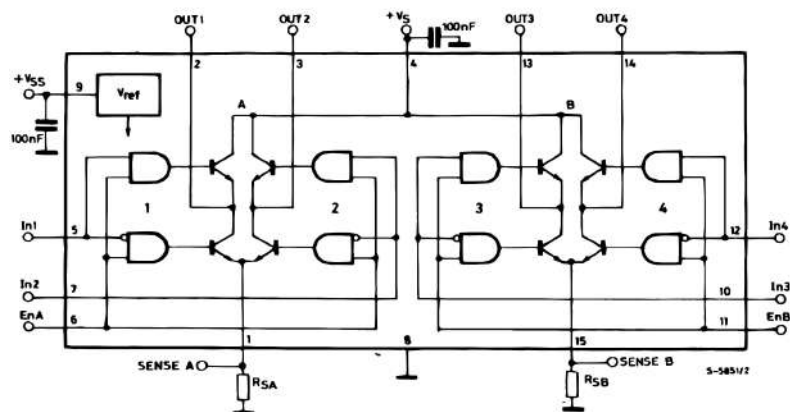


Figura 23: Esquema circuito integrado puente H

5.1.3 ESP32-WROOM-32

La ESP32-WROOM-32 es un microcontrolador, que cuenta con conectividad Wi-Fi y Bluetooth integrada. En este proyecto, se ha utilizado como unidad principal de control para gestionar el funcionamiento de los motores de corriente continua, la lectura de los encoders y la comunicación inalámbrica con la aplicación móvil.

La versión que hemos utilizado es una placa DevKit, que incluye el chip ESP32-WROOM-32 montado sobre una placa que incorpora todos los componentes necesarios para facilitar su uso: regulador de voltaje, conversor USB-serie, botón de reset y acceso a todos los pines de entrada/salida (GPIO).

La programación de la ESP32 se ha realizado utilizando el entorno Arduino IDE, configurado con la librería específica para placas ESP32. Este entorno permite programar el microcontrolador en lenguaje C/C++.

A continuación se incluye el esquema de pines correspondiente a la ESP32-WROOM-32 que hemos utilizado.

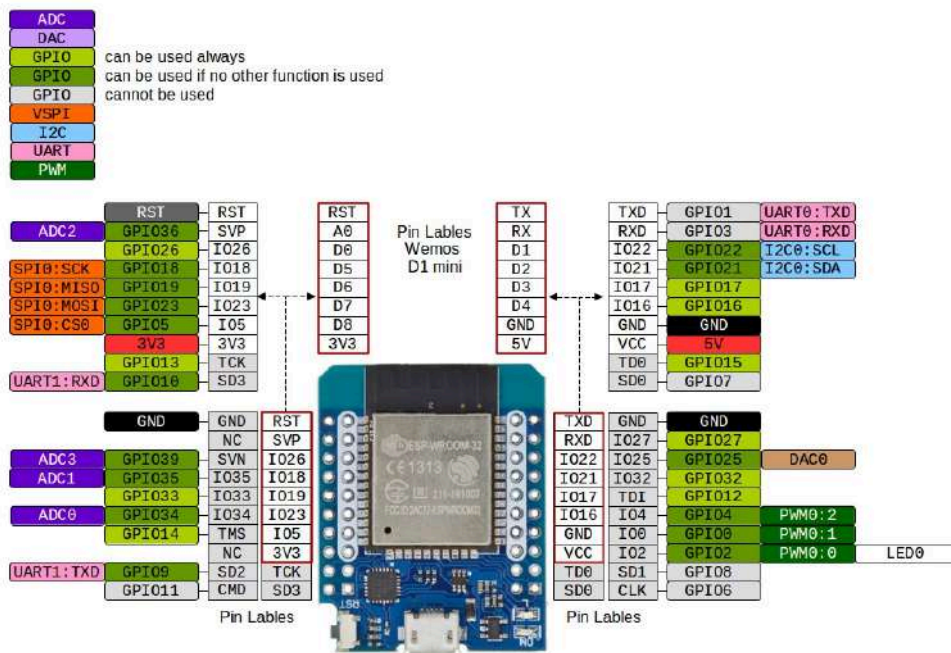


Figura 24: Esquema pines ESP32

5.1.4 PCB

En el desarrollo del presente proyecto, se tomó la decisión de diseñar y fabricar una PCB (Printed Circuit Board o placa de circuito impreso) propia con el objetivo de optimizar la integración y distribución del sistema electrónico del brazo robótico. Esta elección no solo responde a criterios funcionales y de orden, sino que también aporta originalidad y valor diferencial frente a otros enfoques más convencionales basados en cableado libre o protoboards.

Una PCB es una placa rígida fabricada en un material aislante, generalmente fibra de vidrio, sobre la cual se graban pistas de cobre que permiten interconectar componentes electrónicos de forma ordenada, precisa y duradera. Estas pistas reemplazan los cables sueltos y permiten una integración más limpia, fiable y profesional de los distintos módulos electrónicos.

Ventajas y motivos de su implementación

La implementación de una PCB en este proyecto se justificó por varios motivos clave:

- Optimización del espacio interno: La placa fue diseñada específicamente en función de la geometría del brazo robótico, permitiendo alojar de forma compacta los drivers de los motores, la alimentación, los conectores y las señales de control.

- Reducción del desorden de cables: Uno de los principales problemas en montajes con múltiples motores y sensores es el desorden de conexiones, lo que puede dificultar tanto el montaje como el mantenimiento. La PCB agrupa todas las conexiones en una única plataforma organizada, minimizando la probabilidad de errores y cortocircuitos.

- Facilidad de montaje y conexión: Gracias a la disposición lógica de los componentes y conectores en la PCB, la conexión de los distintos elementos del sistema (motores, controladores, alimentación, señales desde Arduino, etc.) se realizó de forma clara, rápida y sin ambigüedades.

- Mayor fiabilidad eléctrica: Las conexiones soldadas sobre la PCB ofrecen una mayor estabilidad eléctrica frente a los cables sueltos o conectores provisionales, reduciendo el riesgo de falsos contactos o pérdidas de señal durante el funcionamiento del brazo.

- Valor añadido y originalidad: La fabricación de una PCB a medida no solo aporta funcionalidad al proyecto, sino que también supone una diferenciación significativa respecto a otros grupos. Mientras que la mayoría de los proyectos similares optan por el uso de protoboards o conexiones por jumper cables, esta solución demuestra un mayor nivel de desarrollo técnico, planificación y

personalización.

Contribución al conjunto del proyecto

La PCB cumple un papel central en el sistema de control del brazo robótico, actuando como una plataforma de interconexión compacta que permite integrar todos los elementos electrónicos de forma segura y ordenada. Además, se diseñó para minimizar la intervención humana posterior, asegurando que la electrónica pudiera mantenerse protegida dentro de la estructura del brazo y reduciendo significativamente la necesidad de ajustes o correcciones durante la operación.

Esta decisión de diseño demuestra una apuesta por la eficiencia, la estética funcional y la robustez técnica, alineándose con los objetivos del proyecto tanto a nivel académico como práctico. Asimismo, este enfoque fomenta el aprendizaje de técnicas de diseño profesional de circuitos, como el uso de software de diseño electrónico (por ejemplo, KiCad o Eagle), la planificación de pistas, y la fabricación o encargo de placas.

El diseño de la PCB desarrollado para este proyecto ha sido totalmente personalizado en función de las necesidades concretas del brazo robótico. Desde la fase de planificación, se priorizó la claridad estructural, la accesibilidad de los conectores y la compatibilidad con la geometría interna del brazo, adaptando el tamaño y disposición de la placa para que pudiera ser integrada sin comprometer el funcionamiento mecánico.

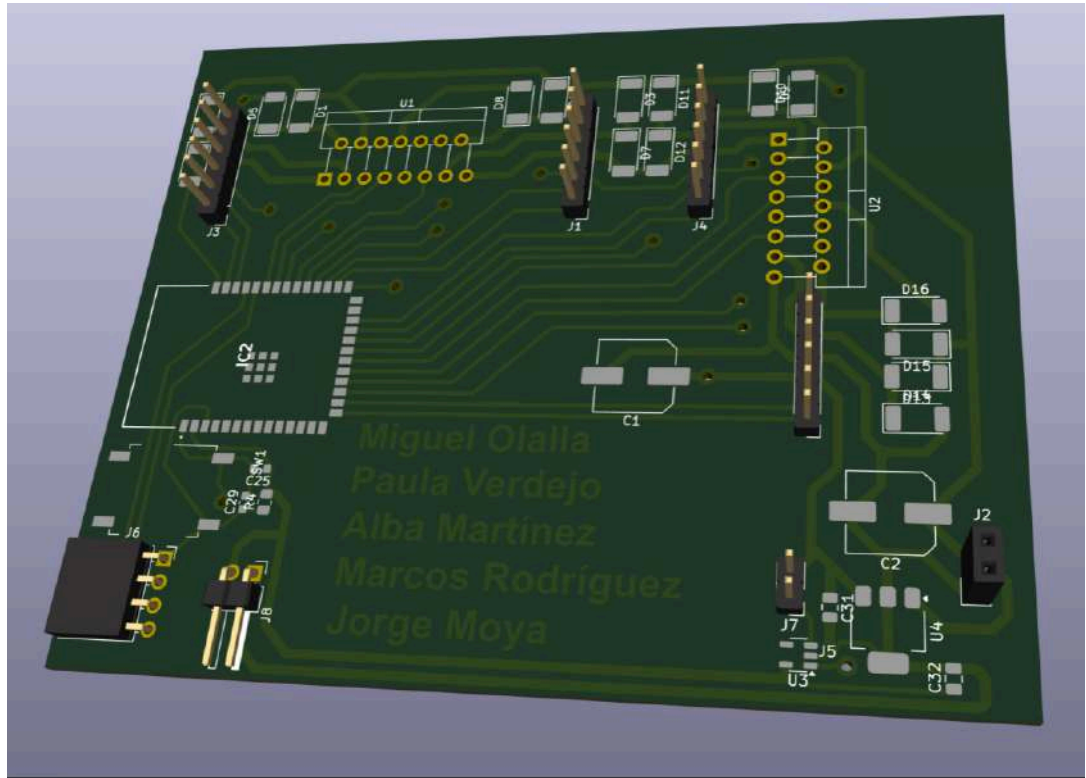
A continuación se incluyen imágenes del diseño de la PCB:



Figura 25: Diseño de la PCB

En la imagen se muestra el diseño final de la PCB personalizada desarrollada para el proyecto del brazo robótico. Esta placa fue diseñada desde cero utilizando el software KiCad, con el objetivo de integrar de forma ordenada todos los componentes

electrónicos necesarios y minimizar el uso de cableado externo. En el centro destaca el driver L298N, acompañado de diodos de protección. A la izquierda se ubica el zócalo para el microcontrolador, y a la derecha los conectores de salida hacia los motores y la fuente de alimentación. Se han utilizado pistas anchas para las líneas de potencia y un correcto ruteado de las señales para garantizar una estructura eficiente y sin interferencias. Además, se ha añadido una serigrafía con los nombres de los integrantes del grupo como elemento identificativo. Este diseño no solo mejora la organización del sistema electrónico, sino que también



representa un aporte original y diferenciador respecto a otros proyectos.

Figura 26: Diseño de la PCB

Una vez impresas las capas de cobre y aislante, se procedió al revelado, grabado y taladrado de la placa. Posteriormente, se realizó el estañado manual de las pistas para asegurar una mejor conductividad y durabilidad de las conexiones. Finalmente, se soldaron todos los componentes necesarios y se verificó su correcto funcionamiento mediante pruebas funcionales.

Como detalle distintivo, está PCB incluye impreso el nombre de todos los integrantes del grupo en su superficie, lo cual no solo refuerza el carácter original y personalizado del trabajo, sino que simboliza el compromiso y la autoría completa del desarrollo electrónico dentro del proyecto.



Figura 27: Impresión de la PCB, proceso de baño en HCL, H2O2 y agua destilada y resultado

El resultado final de la PCB con los componentes añadidos quedaría de la siguiente forma:

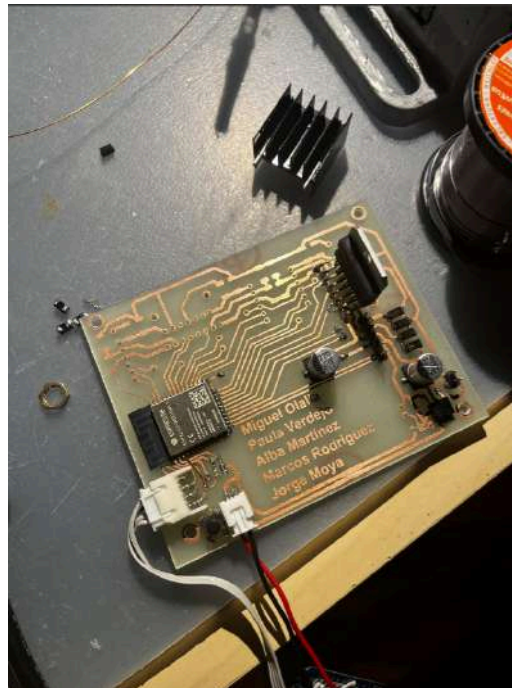


Figura 28: PCB terminada

5.2 Implementación de los motores

5.2.1 Pruebas iniciales y conexiones

Antes de implementar el control definitivo, se realizaron pruebas básicas para verificar el correcto funcionamiento de los **motores de CC**, el **punto H** y el **encoder**.

Para ello, se comenzó conectando un solo motor DC al punto H L298N y a la ESP32-WROOM-32, con la siguiente configuración:

IN 1 → I00

IN 2 → I02

GND → GND esp32

ENA → I025

RED-Motor Power (+) → OUT 1

BLACK-Motor Power (-) → OUT 2

Quad encoder Ground → GND

Quad encoder +5V Vcc → Vcc:5V

Quad encoder A signal → I021

Quad encoder B signal → I022

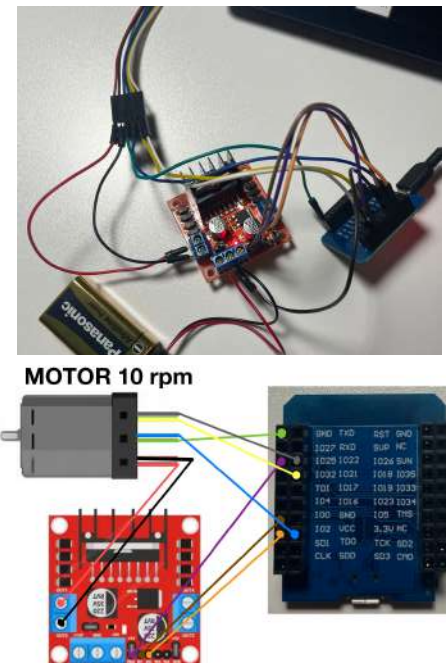


Figura 29: Conexiones de un motor

Estas conexiones junto al código de prueba, permitieron comprobar:

- El giro del motor en ambos sentidos.
- El control de velocidad mediante modulación PWM.
- El funcionamiento del encoder, visualizando los pulsos en el *Serial Monitor*.

Inicialmente, se detectaron errores de compilación al cargar el código, debido a un conflicto con la versión del núcleo de la placa en el Gestor de tarjetas de Arduino IDE. Para resolverlo, se instaló manualmente la versión **2.0.14** de **esp32 by Espressif Systems**, tras lo cual el código se compiló y ejecutó correctamente.

5.2.2 Realimentación

Una vez comprobado el funcionamiento básico del motor y del encoder, se procedió a implementar el sistema de realimentación para conocer el desplazamiento angular del eje en tiempo real. Esta etapa es esencial para poder aplicar posteriormente controladores en lazo cerrado, como el PID.

La lectura se implementó mediante una interrupción en el canal A del encoder. Cada vez que se detecta un flanco de subida en dicha señal, se ejecuta la función (`readEncoder()`) que lee el estado actual del canal B. Según ese estado, se determina el sentido de giro y se incrementa o decrementa el contador global de pasos.

```
void readEncoder() {
    int b = digitalRead(encoderPinB);
    if (b > 0) {
        encoderCount++;
    } else {
        encoderCount--;
    }
}
```

Con esta función, se obtiene una lectura precisa y en tiempo real del desplazamiento del motor. El valor del contador `encoderCount` representa la posición relativa del motor, y puede transformarse en ángulos o vueltas completas.

El encoder integrado en el motor genera 11 pulsos por cada vuelta del eje del motor interno. En el caso del motor de 10 rpm, la reductora interna tiene una relación de 1:600, por lo que 1 vuelta en el eje de salida equivale a 600 vueltas del eje interno, que equivale a 6600 pulsos.

A esto se suma la transmisión por engranajes utilizada en la base del robot, con una relación 40:60, lo que introduce un factor de multiplicación de 1.5.

Por tanto, una vuelta completa del eje de salida de la base equivale a 9900 pulsos.

En el caso de la base, no se ha puesto ninguna transmisión por engranajes.

Para el caso del motor de 5 rpm, el encoder integrado genera también 11 pulsos, sin embargo, la reductora interna tiene un relación de 1:900 por lo que 1 vuelta del eje de salida, equivale a 9900 pulsos, y añadiendo, la transmisión por engranajes empleada en el hombro y el codo: 16:60, finalmente se generan 37125 pulsos por 1 vuelta en el eje de salida.

Este sistema de realimentación es la base para el posterior cálculo del error en el controlador PID.

5.2.3 PID

El algoritmo **PID** (Proporcional, Integral, Derivativo) se basa en la suma de tres componentes que actúan de forma complementaria para corregir el error entre una referencia y una señal medida. La expresión matemática de un controlador PID es la siguiente:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(t) \cdot dt + K_d \cdot \frac{de}{dt}$$

Figura 30: Fórmula PID

Donde:

- e(t) es el error en el instante de tiempo t
- K_p, K_i, K_d son los coeficientes de las acciones proporcional, integral y derivativa.
- u(t) es la salida del controlador.

Cada una de las tres acciones del PID contribuye de forma distinta al comportamiento global del sistema:

- La parte **proporcional (P)** actúa sobre el error actual, corrigiendo en función de su magnitud.
- La parte **integral (I)** acumula los errores pasados, ayudando a eliminar el error en régimen permanente.
- La parte **derivativa (D)** anticipa el error futuro a partir de su velocidad de cambio, mejorando la estabilidad.

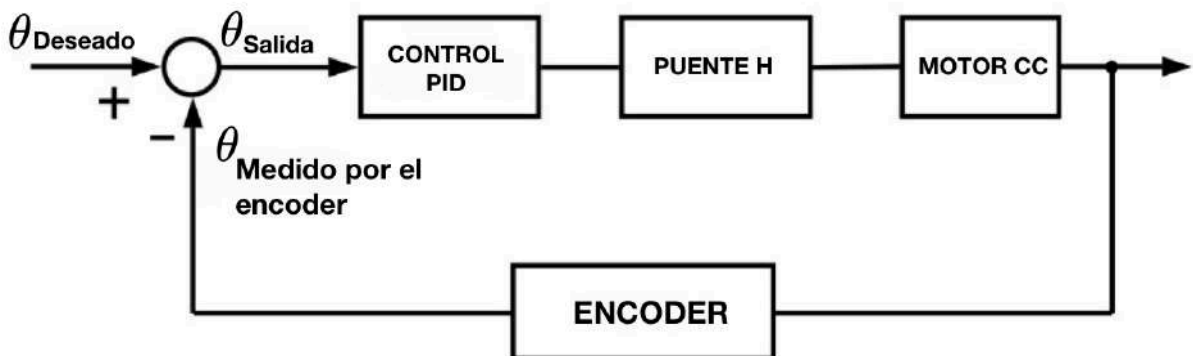


Figura 30: Lazo de Realimentación

Primero lo probamos indicando nosotros el ángulo deseado.
El PID para el motor de 10 rpm:


```

1  #include "driver/ledc.h" // Librería de ESP32 para generar PWM
2
3  // ----- Pines -----
4  const int encoderPinA = 22; // Canal A del encoder
5  const int encoderPinB = 21; // Canal B del encoder (detecta dirección)
6
7  const int IN1 = 0;
8  const int IN2 = 2;
9  const int ENA = 25;
10
11 // ----- Constantes del motor -----
12 const int pulsesPerTurn = 11 * 600 * 1.5;
13
14
15 // ----- Variables -----
16 volatile long encoderCount = 0; // Contador de pulsos del encoder (posición absoluta)
17 float motorPositionDegrees = 0; // Posición del eje en grados
18
19 float targetDegrees = -50.0; // Posición deseada del motor
20 long targetPositionPulses = 0; // Posición deseada en pulsos (se calcula a partir de grados)
21
22 // PID
23 float proportional = 2; // Constante proporcional (Kp)
24 float integral = 0.00005; // Constante integral (Ki)
25 float derivative = 0.01; // Constante derivativa (Kd)
26
27 float controlSignal = 0; // Salida del PID (valor de PWM con signo)
28 float previousError = 0; // Error anterior (para calcular derivada)
29 float errorIntegral = 0; // Suma acumulada del error (parte integral)
30 unsigned long previousTime = 0; // Tiempo anterior (para calcular deltaTime)
31
32 // PWM settings
33 const int pwmChannel = 0; // Canal de PWM hardware
34 const int pwmFreq = 5000; // Frecuencia del PWM (5 kHz)
35 const int pwmResolution = 8; // Resolución de 8 bits (0-255)
36
37 // ----- Setup -----
38 void setup() {
39     Serial.begin(115200);
40
41     // Configuración de entradas del encoder
42     pinMode(encoderPinA, INPUT);
43     pinMode(encoderPinB, INPUT);
44     attachInterrupt(digitalPinToInterrupt(encoderPinA), readEncoder, RISING);
45     // Interrupción: cada flanco de subida en A se llama a readEncoder()
46
47     // Pines de control del motor
48     pinMode(IN1, OUTPUT);
49     pinMode(IN2, OUTPUT);
50
51     ledcSetup(pwmChannel, pwmFreq, pwmResolution);
52     ledcAttachPin(ENA, pwmChannel);
53
54     // de grados deseados en pulsos objetivo
55     targetPositionPulses = (targetDegrees / 360.0) * pulsesPerTurn;
56
57     previousTime = micros(); // Iniciamos el tiempo para cálculo de derivadas
58 }
59

```

```

60 // ----- Loop -----
61 void loop() {
62     calculatePID(); // Calculamos nueva señal PID
63     driveMotor();   // Aplicamos la señal al motor
64     printPositions(); //
65     delay(50);      // Refrescamos cada 50 ms (20 Hz de control)
66 }
67
68 // ----- Leer Encoder -----
69 void readEncoder() {
70     //dirección del motor en función del estado del canal B
71     int b = digitalRead(encoderPinB);
72     if (b > 0) {
73         encoderCount++; // Giro en un sentido
74     } else {
75         encoderCount--; // Giro en el otro sentido
76     }
77 }
78
79 // ----- PID -----
80 void calculatePID() {
81     unsigned long currentTime = micros();
82     float deltaTime = (currentTime - previousTime) / 1e6; // Tiempo en segundos
83     previousTime = currentTime;
84
85     // Cálculo del error entre objetivo y actual
86     float error = (float)(targetPositionPulses - encoderCount);
87
88     // Derivada: variación del error en el tiempo
89     float errorDerivative = (error - previousError) / deltaTime;
90
91     // Integral: acumulación del error en el tiempo
92     errorIntegral += error * deltaTime;
93
94     // Control PID final
95     controlSignal =
96         proportional * error +
97         integral * errorIntegral +
98         derivative * errorDerivative;
99
100     previousError = error;
101 }
102
103 // ----- Control del Motor -----
104 void driveMotor() {
105     int pwm = (int)fabs(controlSignal); // Valor absoluto para intensidad del PWM
106
107     // Limitar el rango
108     if (pwm > 255) pwm = 255;
109     if (pwm < 30 && pwm > 0) pwm = 30; // Evitar zonas muertas del motor
110
111     // Dirección del motor
112     if (controlSignal > 0) {
113         digitalWrite(IN1, HIGH);
114         digitalWrite(IN2, LOW);
115     } else if (controlSignal < 0) {
116         digitalWrite(IN1, LOW);
117         digitalWrite(IN2, HIGH);
118     } else {
119         digitalWrite(IN1, LOW);
120         digitalWrite(IN2, LOW);
121     }
122
123     ledcWrite(pwmChannel, pwm); // Aplicamos PWM
124 }
125
126 // ----- Mostrar por Serial -----
127 void printPositions() {
128     // Convertimos la posición del motor a grados
129     motorPositionDegrees = ((float)encoderCount / pulsesPerTurn) * 360.0;
130
131     Serial.print("Objetivo: ");
132     Serial.print(targetDegrees);
133     Serial.print(" ° | Actual: ");
134     Serial.print(motorPositionDegrees);
135     Serial.print(" ° | PWM: ");
136     Serial.println(controlSignal);
137 }

```

En el caso del **motor de 10 rpm** utilizado en la base del robot, se emplearon los siguientes parámetros para el controlador PID:

$$K_p=2.0 \quad K_i=0.00005 \quad K_d=0.01$$

Estos valores fueron ajustados de forma experimental, observando el comportamiento del sistema durante las pruebas. También se tuvo en cuenta que el motor dispone de una reducción interna de 1:600 y de una transmisión por engranajes 40:60, lo que da lugar a un sistema mecánicamente lento, pero con un par elevado. Por tanto:

-El valor **proporcional** permite una corrección eficiente del error sin generar oscilaciones.

-El valor **integral**, muy bajo, evita la acumulación excesiva de error que podría provocar sobre oscilaciones o inestabilidad.

-El término **derivativo** actúa suavemente, ayudando a frenar el sistema a medida que se aproxima al objetivo, mejorando la estabilidad sin amplificar el ruido del encoder.

Con esta configuración, el motor alcanzó la posición deseada con una respuesta precisa, suave y sin oscilaciones significativas, lo que permitió su integración efectiva en el sistema robótico.

Objetivo: -32.16 ° | Actual: -32.29 ° | PWM: 0.00

Figura 31: Serial Monitor PID

Para el caso de la pala se ha usado: $K_p=1.0$ $K_i=0.00001$ $K_d=0.00$
Puesto que no dispone de transmisión por engranajes.

En el caso del **motor de 5 rpm**, utilizado en las articulaciones del **hombro y el codo**, se emplearon los siguientes parámetros PID:

$$K_p=1.0 \quad K_i=0.00001 \quad K_d=0.001$$

Este motor dispone de una reducción interna de 1:900 y una transmisión mecánica por engranajes de relación 16:60, lo que da lugar a un sistema aún más lento que el motor de 10 rpm, pero con un par considerablemente mayor, necesario para levantar la estructura del brazo.

Se mantuvo el mismo valor proporcional que en el motor de 10 rpm, ya que la ganancia K_p ofrecía una corrección eficaz del error sin provocar oscilaciones.

Se redujo el valor de K_i a **0.00001**, ya que al tratarse de un sistema con respuesta lenta, la acción integral tiende a acumularse con mayor facilidad. Un valor demasiado alto provocaría oscilaciones o incluso inestabilidad.

También se ajustó el valor de K_d a **0.001**, reduciendo la acción derivativa para evitar una sobrecompensación del sistema ante pequeñas variaciones en el error, lo que mejoró la suavidad de la respuesta sin amplificar el ruido del encoder.

6. APLICACIÓN

6.1. Descripción general

La aplicación Android **Robótica 3** ha sido desarrollada como interfaz gráfica de usuario para controlar un brazo robótico basado en ESP32. Permite enviar coordenadas tridimensionales desde un dispositivo móvil al microcontrolador a través de una red WiFi local, o unos ángulos para el control de los motores. La app se comunica mediante el protocolo **TCP/IP** y envía posiciones objetivo en formato cartesiano para que el sistema embebido realice el movimiento correspondiente usando cinemática inversa y control PID.

6.2. Funcionalidades

Conexión WiFi:

El usuario se conecta automáticamente a la misma red local que el ESP32, utilizando la dirección IP proporcionada por el microcontrolador.

Entrada de coordenadas/ entrada de ángulos

La interfaz incluye tres campos de texto (x, y) donde el usuario introduce la posición deseada en milímetros o entrada de ángulos a través de un botón slider.

Comunicación TCP:

Al pulsar el botón **ENVIAR**, la aplicación establece una conexión con el ESP 32 y transmite un mensaje con el siguiente formato: (x, y) o **base:90**.

Recepción de respuesta:

La app muestra en pantalla la respuesta del servidor, que puede ser:

“**OK**” si las coordenadas son válidas y alcanzables.

“**Fuera de alcance**” si el punto excede las capacidades físicas del brazo.

“**Formato inválido**” si los datos no siguen la estructura esperada.

6.3. Tecnologías utilizadas

Lenguaje: Kotlin

Entorno: Android Studio

Protocolo de comunicación: TCP/IP

Red: WiFi local

6.4. Códigos implementados

KOITLIN:

El código en Kotlin se encarga de la lógica de red y procesamiento de datos, permitiendo que la aplicación Android se comuniquen con un servidor remoto en este caso, un ESP32 que controla un brazo robótico mediante una conexión TCP/IP por WiFi.

Funciones clave pantalla 1:

1. **Leer coordenadas del usuario (x, y)** desde los campos de entrada en la interfaz (cuando el usuario pulsa un botón).
2. **Establecer una conexión TCP** con la IP del ESP32.
3. **Enviar un mensaje en texto plano** con el formato "**x, y**" al ESP32.
4. **Esperar y recibir una respuesta del servidor**, como "OK", "Fuera de alcance", etc.
5. **Mostrar la respuesta al usuario**

Funciones clave pantalla 2:

6. **Leer ángulo robot cada eje** desde los campos de entrada en la interfaz (cuando el usuario pulsa un botón).
7. **Establecer una conexión TCP** con la IP del ESP32.
8. **Enviar un mensaje al arduino**
9. **Esperar y recibir una respuesta del servidor**, como "OK", "Fuera de alcance", etc.

ANDROID MANIFEST.XML

Es un archivo XML que le dice al sistema operativo Android cómo debe comportarse la aplicación, qué necesita para funcionar y qué componentes contiene.

1. Declara los componentes de la app

Define qué actividades (Activity), servicios (Service), receptores (BroadcastReceiver) y proveedores (ContentProvider) existen en tu app.

2. Permisos

Indica qué permisos necesita tu app para funcionar (como acceder a internet, cámara, ubicación, etc.).

Para comunicación con ESP32 por WiFi, se necesita:

```
<uses-permission android:name="android.permission.INTERNET" />
```

3. Información general de la app

- Nombre del paquete (package), ícono, nombre de la app, tema, versión, etc.

4. Configura el punto de entrada

Define cuál es la actividad principal que se abre cuando se lanza la app:

```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
```

5. Compatibilidad y restricciones

- Establece versiones mínimas de Android, hardware requerido, etc.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest
xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET"
/>
    <uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.ROBITOCA3"
        tools:targetApi="31">
```



```

        <!-- Esta es la actividad principal que se lanza al abrir
la app -->
        <activity
            android:name=".FirstAppActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"
/>
                <category
android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <!-- Otra actividad, pero sin intent-filter -->
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.ROBITOCA3" />
    </application>

</manifest>

```

.XML

En Android Studio, un archivo .xml sirve principalmente para definir estructuras de datos y configuraciones, no para lógica de programación. Se usa mucho para interfaz de usuario (UI) y para configuraciones del sistema o red.

PROGRAMA

PANTALLA 1

Para este programa hemos programado dos pantallas, la primera de ellas pasa unas coordenadas al robot que se seleccionan en un mapa, esta es FIRST ACTIVITY.



FIRST ACTIVITY.kt

```
package com.ePRUEBA.robistica3

import android.content.Intent
import android.os.Bundle
import android.os.StrictMode
import android.view.MotionEvent
import android.view.View
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.io.PrintWriter
import java.net.Socket
import kotlin.concurrent.thread

class FirstAppActivity : AppCompatActivity() {
```

```

override fun onCreate(savedInstanceState: Bundle?) {

    super.onCreate(savedInstanceState)

    setContentView(R.layout.activity_first_app)

    val tvCoordinates =
findViewById<TextView>(R.id.tvCoordinates)

    val blockedArea = findViewById<View>(R.id.blocked_area)

    val btnIrPantallaNueva =
findViewById<Button>(R.id.btnIrPantallaNueva)

    // Botón para cambiar de pantalla
    btnIrPantallaNueva.setOnClickListener {

        val intent = Intent(this, PantallaNueva::class.java)

        startActivity(intent)

    }

    // Detectar toques en la pantalla
    tvCoordinates.setOnTouchListener { v, event ->

        if (event.action == MotionEvent.ACTION_DOWN) {

            v.performClick()

            val screenHeight = tvCoordinates.height

            val x_px = event.x

            val y_px = screenHeight - event.y

            val dpi =
resources.displayMetrics.densityDpi.toFloat()

            val x_m = (x_px / dpi) * 0.0254f

```

```

val y_m = (y_px / dpi) * 0.0254f

val location = IntArray(2)

blockedArea.getLocationOnScreen(location)

val boxX = location[0]

val boxY_top = location[1]

val boxWidth = blockedArea.width

val boxHeight = blockedArea.height

val boxY = screenHeight - (boxY_top + boxHeight)

val dentroDelCuadro = x_px.toInt() in boxX..(boxX
+ boxWidth) &&

    y_px.toInt() in boxY..(boxY + boxHeight)

    if (!dentroDelCuadro) {

        tvCoordinates.text = String.format("X: %.3f
m, Y: %.3f m", x_m, y_m)

        sendCoordinatesToESP32("X:$x_m,Y:$y_m\n")

    }

    true

} else {

    false

}

}

}

```

```

private fun sendCoordinatesToESP32(message: String) {

StrictMode.setThreadPolicy(StrictMode.ThreadPolicy.Builder().per
mitAll().build())

    thread {

        try {

            val socket = Socket("192.168.223.8", 80)

            val writer =
PrintWriter(socket.getOutputStream(), true)

            writer.println(message)

            writer.flush()

            socket.close()

            runOnUiThread {

                Toast.makeText(this, "Mensaje enviado al
ESP32", Toast.LENGTH_SHORT).show()

            }

        } catch (e: Exception) {

            e.printStackTrace()

            runOnUiThread {

                Toast.makeText(this, "Error al conectar con
ESP32", Toast.LENGTH_SHORT).show()

            }

        }

    }

}
}

```

FIRSTACTIVITY.XML

```

<FrameLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- Fondo -->
    <ImageView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:contentDescription="@string/fondo_desc"
        android:scaleType="centerCrop"
        android:src="@drawable/fondo" />

    <!-- Área bloqueada -->
    <View
        android:id="@+id/blocked_area"
        android:layout_width="200dp"
        android:layout_height="150dp"
        android:layout_gravity="bottom|start"
        android:background="@android:color/darker_gray"
        android:clickable="false"
        android:focusable="false" />

    <!-- Texto con coordenadas -->
    <TextView
        android:id="@+id/tvCoordinates"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:gravity="center"
        android:text="@string/toca_pantalla"
        android:textSize="40sp"
        android:textColor="@android:color/black"
        android:textAllCaps="true"
        android:fontFamily="@font/alfaslab" />

    <!-- Botón para ir a pantalla nueva -->
    <Button
        android:id="@+id/btnIrPantallaNueva"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|center_horizontal"
        android:layout_marginBottom="30dp"
        android:text="CONTROL MANUAL"
        android:textColor="@android:color/white"
        android:background="#FF0000" />
</FrameLayout>

```


Código arduino IDE

```

1  #include <WiFi.h>
2
3  // ----- CONFIG WIFI -----
4  const char* ssid = "ROBOTICO";      // Nombre de la red WiFi que crea la ESP32
5  const char* password = "12345676";  // Contraseña de la red WiFi
6  WiFiServer server(80);              // Crea un servidor en el puerto 80 (HTTP)
7
8
9  // ----- TAMAÑOS -----
10 const float tablero_lado_largo_m = 0.60;  // Queremos que 15.21 cm del móvil equivalgan a 60 cm reales
11
12 // Tamaño físico real del móvil (calculado a partir de 6.6" y 2340x1080px)
13 const float pantalla_movil_ancho_m = 0.0702; // Ancho de la pantalla 7.02 cm
14 const float pantalla_movil_alto_m = 0.1521;  // Alto de la pantalla 15.21 cm
15
16 // Escala global basada en el lado largo del móvil
17 const float escala_global = tablero_lado_largo_m / pantalla_movil_alto_m; // 60 cm / 15.21 cm = 3.94
18
19 void setup() {
20   Serial.begin(115200);
21   WiFi.begin(ssid, password); //inicia red wifi
22
23   while (WiFi.status() != WL_CONNECTED) {
24     delay(1000);
25     Serial.println("Conectando a WiFi...");
26   }
27
28   Serial.println("Conectado a WiFi.");
29   Serial.print("Dirección IP: ");
30   Serial.println(WiFi.localIP()) //muestra la IP local para la conexión;
31
32   server.begin();
33 }
34
35 void loop() {
36   WiFiClient client = server.available(); //espera a la app
37   if (client) {
38     Serial.println("Cliente conectado.");
39     while (client.connected()) {
40       if (client.available()) {
41         String mensaje = client.readStringUntil('\n');
42         mensaje.trim();
43         //la app envía:
44         Serial.print("Recibido: ");
45         Serial.println(mensaje);
46
47         float x_m = 0, y_m = 0;
48         int xIndex = mensaje.indexOf("X:");
49         int yIndex = mensaje.indexOf(",Y:");
50
51         if (xIndex != -1 && yIndex != -1) {
52           String xStr = mensaje.substring(xIndex + 2, yIndex);
53           String yStr = mensaje.substring(yIndex + 3);
54
55           x_m = xStr.toFloat(); //X en metros, relativos a la tablet
56           y_m = yStr.toFloat(); //Y en metros relativos a la tablet
57
58           // ----- ESCALADO GLOBAL -----
59
60           float x_escalado = x_m * escala_global;
61           float y_escalado = y_m * escala_global;
62
63           // Mostrar por serial, escala para convertir valores del móvil a metros
64           Serial.print("Escala (global) -> X: ");
65           Serial.print(x_escalado, 3);
66           Serial.print(" m, Y: ");
67           Serial.print(y_escalado, 3);
68           Serial.println(" m");

```

```

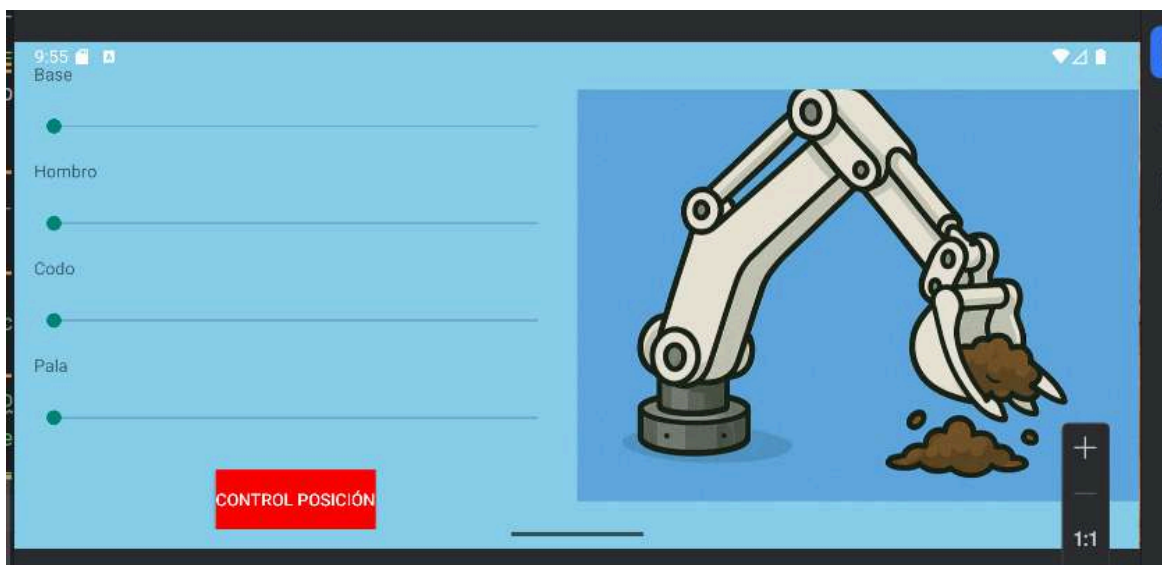
69
70 // Respuesta de la app
71 client.println("X tablero: " + String(x_escalado, 3) + " m, Y tablero: " + String(y_escalado, 3) + " m");
72 } else {
73   client.println("Formato inválido");
74 }
75 }
76 }
77 client.stop(); //finalizas la conexión con la app
78 Serial.println("Cliente desconectado.");
79
80 }

```

PANTALLA 2

La segunda pantalla se utiliza para mover los motores y se llama PANTALLA NUEVA.

Ambas pantallas se comunican entre ellas mediante los botones rojos, que permiten ir de una pantalla a la otra, cambiando el cambio de comunicación con el robot.



PANTALLANUEVA.KT

```

// PantallaNueva.kt
package com.ePRUEBA.robótica3

import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.os.StrictMode
import android.widget.Button
import android.widget.SeekBar

```

```

import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import java.io.PrintWriter
import java.net.Socket
import kotlin.concurrent.thread

class PantallaNueva : AppCompatActivity() {

    private val handler = Handler(Looper.getMainLooper())
    private val delayMillis = 200L
    private val delayMap = mutableMapOf<String, Runnable>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.pantallanueva)

        val btnVolver = findViewById<Button>(R.id.btnVolverAFirst)
        btnVolver.setOnClickListener {
            finish()
        }

        val sliders = mapOf(
            "Base" to findViewById<SeekBar>(R.id.sliderBase),
            "Hombro" to findViewById<SeekBar>(R.id.sliderHombro),
            "Codo" to findViewById<SeekBar>(R.id.sliderCodo),
            "Pala" to findViewById<SeekBar>(R.id.sliderPala)
        )

        for ((nombre, slider) in sliders) {
            slider.max = 360

            slider.setOnSeekBarChangeListener(object :
SeekBar.OnSeekBarChangeListener {
                override fun onProgressChanged(seekBar: SeekBar?,
progress: Int, fromUser: Boolean) {
                    val valorGrados = progress - 180
                    val mensaje = "$nombre:$valorGrados"

                    Toast.makeText(this@PantallaNueva, mensaje,
Toast.LENGTH_SHORT).show()

                    delayMap[nombre]?.let {
handler.removeCallbacks(it) }

                    val runnable = Runnable {
                        sendToESP32(mensaje)
                    }
                    delayMap[nombre] = runnable
                }
            })
        }
    }
}

```

```

        handler.postDelayed(runnable, delayMillis)
    }

    override fun onStartTrackingTouch(seekBar:
SeekBar?) {}
    override fun onStopTrackingTouch(seekBar:
SeekBar?) {}
    })
    }

    private fun sendToESP32(message: String) {

StrictMode.setThreadPolicy(StrictMode.ThreadPolicy.Builder().perm
itAll().build())

        thread {
            try {
                val socket = Socket("192.168.223.8", 80)
                val writer = PrintWriter(socket.getOutputStream(),
true)

                writer.println(message)
                writer.flush()
                socket.close()

                runOnUiThread {
                    Toast.makeText(this, "Ángulo enviado:
$message", Toast.LENGTH_SHORT).show()
                }
            } catch (e: Exception) {
                e.printStackTrace()
                runOnUiThread {
                    Toast.makeText(this, "Error al conectar con
ESP32", Toast.LENGTH_SHORT).show()
                }
            }
        }
    }
}

```

PANTALLANUEVA.XML

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal"
    android:background="#87CEEB">

```

```

<!-- COLUMNA IZQUIERDA: Sliders + botón -->
<LinearLayout
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_weight="1"
    android:orientation="vertical"
    android:padding="16dp">

    <!-- Base -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="Base" />

    <SeekBar
        android:id="@+id/sliderBase"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="2"
        android:max="360" />

    <!-- Hombro -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="Hombro" />

    <SeekBar
        android:id="@+id/sliderHombro"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="2"
        android:max="360" />

    <!-- Codo -->
    <TextView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:text="Codo" />

    <SeekBar
        android:id="@+id/sliderCodo"
        android:layout_width="match_parent"
        android:layout_height="0dp"

```

```

        android:layout_weight="2"
        android:max="360" />

<!-- Pala -->
<TextView
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:text="Pala" />

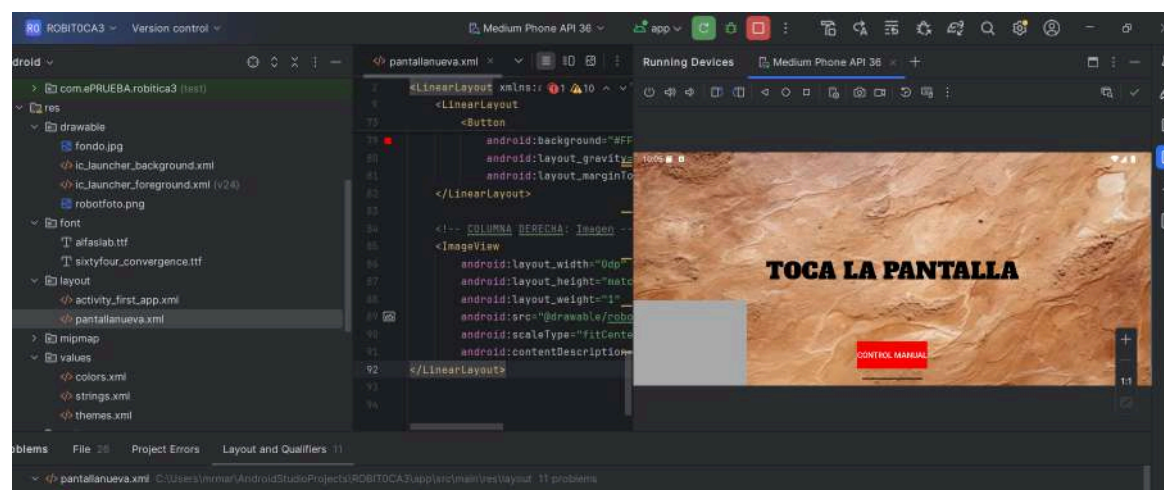
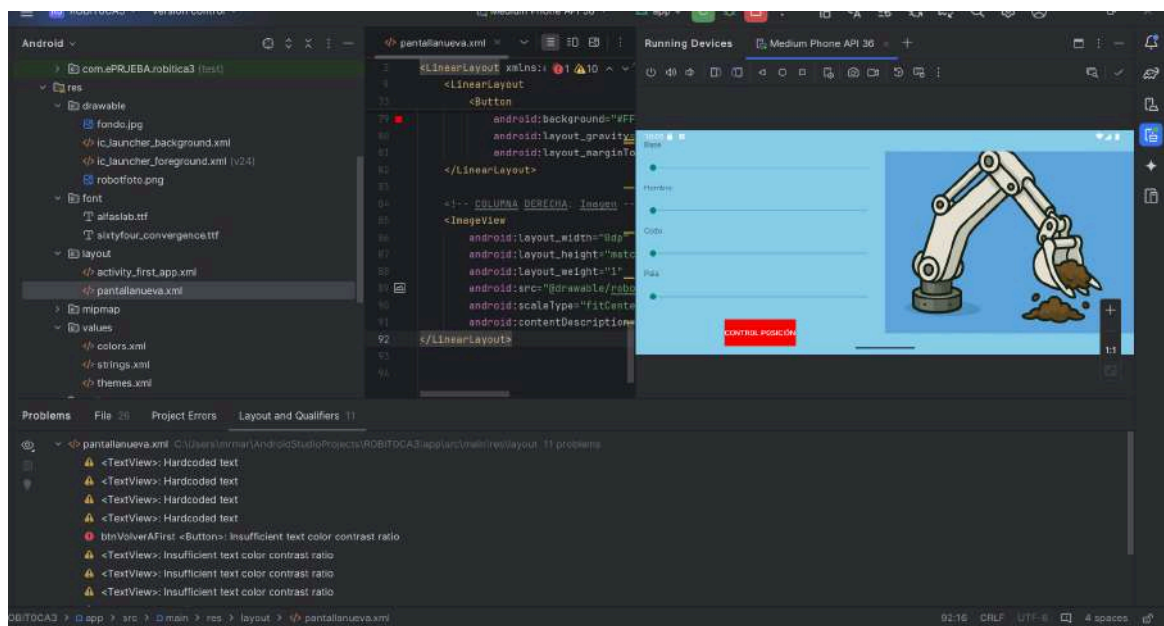
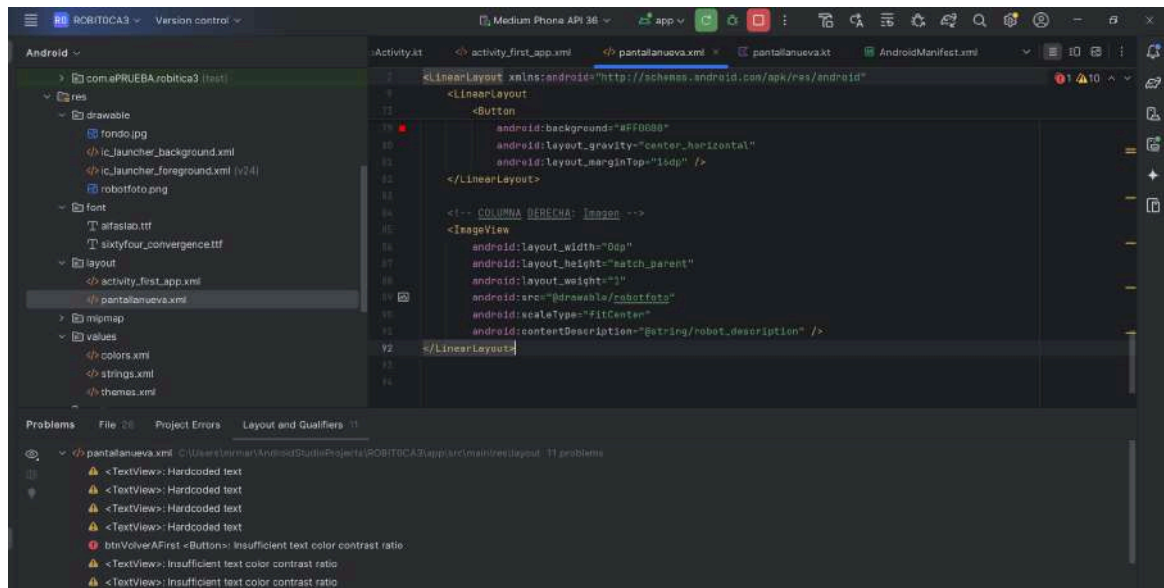
<SeekBar
    android:id="@+id/sliderPala"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="2"
    android:max="360" />

<!-- Botón rojo -->
<Button
    android:id="@+id/btnVolverAFirst"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="CONTROL POSICIÓN"
    android:textColor="@android:color/white"
    android:background="#FF0000"
    android:layout_gravity="center_horizontal"
    android:layout_marginTop="16dp" />
</LinearLayout>

<!-- COLUMNA DERECHA: Imagen -->
<ImageView
    android:layout_width="0dp"
    android:layout_height="match_parent"
    android:layout_weight="1"
    android:src="@drawable/robotfoto"
    android:scaleType="fitCenter"
    android:contentDescription="@string/robot_description" />
</LinearLayout>

```


VISTAS EN ANDROID STUDIO



7. Fases del montaje

Una vez finalizado el modelado tridimensional del brazo robótico en Fusion 360, se procedió a la fase constructiva del prototipo. Esta etapa comienza con la obtención física de las piezas necesarias, muchas de las cuales fueron impresas en 3D conforme a las dimensiones y geometrías diseñadas previamente en el software.



Figura 32: Piezas impresas en 3D

La primera acción sobre los materiales estructurales consistió en el corte de las barras de aluminio, las cuales conforman los eslabones principales del brazo. Para ello, se tomaron las medidas exactas del modelo 3D y se marcaron sobre las barras con precisión, asegurando la fidelidad dimensional. El corte se realizó manualmente utilizando una sierra adecuada para metales, asegurando un trabajo limpio y controlado.



Figura 33: Corte de las barras de aluminio

Posteriormente, se procedió al lijado de los extremos cortados para eliminar rebabas, mejorar la seguridad del montaje y garantizar un buen ajuste con los componentes que se fijarían a continuación. Una vez acondicionadas, se marcaron las ubicaciones donde irían instalados los motores, tomando como referencia las coordenadas extraídas del diseño CAD. Se realizaron las perforaciones necesarias sobre la barra de aluminio, tanto para los tornillos de fijación de los motores como para permitir el paso del eje del motor.

Cabe destacar que el primer eslabón del brazo (la primera barra de aluminio) alberga en su interior dos motores, cuya integración estructural fue prevista desde el diseño. El segundo eslabón, por su parte, se encarga de accionar la pala mediante un motor adicional, cumpliendo con la función final de recogida de material.

Una vez fabricadas las piezas de resina y mecanizadas las barras de aluminio, se procedió al montaje de las uniones entre los eslabones del brazo robótico. Estas uniones están diseñadas para permitir la rotación precisa de cada segmento en torno a un eje central.

Cada articulación se ensambla insertando una varilla de acero inoxidable a través de una pieza impresa en resina, la cual actúa como soporte y carcasa exterior. En el interior de esta pieza se alojan rodamientos de bolas (tipo 608ZZ) que permiten que la varilla gire con suavidad y mínima fricción. Esta varilla atraviesa también el extremo del eslabón anterior, que ha sido previamente perforado y ajustado para encajar perfectamente con la geometría del conjunto.

La polea dentada, que va solidaria a la varilla, se fija en su posición una vez introducida para asegurar la transmisión de movimiento mediante correa desde el motor interno del eslabón. Finalmente, la base de la pieza impresa se atornilla al extremo del siguiente eslabón (también de aluminio), quedando así ambos tramos firmemente unidos, pero con capacidad de rotación gracias al eje metálico y a los rodamientos.

Este sistema modular facilita tanto el ensamblaje como el mantenimiento del brazo, permitiendo sustituir o reajustar fácilmente cada articulación en caso necesario.



Figura 34: Articulación entre eslabones

Cabe destacar que, durante el montaje del sistema de transmisión del primer eslabón respecto a la base, surgió un inconveniente de diseño. Inicialmente, se había previsto fijar la polea dentada grande directamente a la varilla de acero; sin embargo, esto impedía el movimiento del eslabón, ya que dicha varilla está solidariamente unida a la estructura del propio eslabón. Para resolver este problema, se optó por fijar la polea dentada a la base mediante adhesivo, de modo que la correa pudiera accionar el movimiento del eslabón desde el motor, sin bloquear el giro por un acoplamiento indebido. Esta solución permitió mantener la funcionalidad de la articulación sin comprometer la integridad del conjunto.

La base del brazo robótico se diseñó como una estructura circular de tres capas concéntricas, con el objetivo de ofrecer una sujeción sólida al conjunto y permitir un giro controlado en el eje vertical (rotación base del brazo). La capa inferior fue atornillada directamente a la superficie de trabajo, actuando como soporte estructural.

La capa intermedia tiene la función de alojar un rodamiento de gran tamaño, que permite la rotación suave del conjunto superior con un mínimo rozamiento. Esta solución

mecánica garantiza una base estable pero a la vez móvil, permitiendo que el brazo se oriente hacia distintas direcciones en el plano horizontal.

Finalmente, la capa superior se fijó sobre el rodamiento, sirviendo de plataforma de montaje para el primer eslabón del brazo. Este diseño modular permitió un ensamblaje sencillo y eficiente, manteniendo la alineación entre los componentes.



Figura 35: Base

El movimiento de los distintos eslabones se consigue mediante un sistema de transmisión por correas dentadas, las cuales se conectan directamente al eje de salida de los motores. Esta solución fue seleccionada por su bajo coste, facilidad de implementación y capacidad para transmitir el par necesario con precisión y sin deslizamientos indeseados.

Las poleas fueron colocadas tanto en los ejes de los motores como en los ejes de los eslabones móviles. De este modo, al activarse el motor, la correa transmite el giro a la articulación correspondiente, generando el movimiento del brazo. Esta configuración permitió alojar los motores dentro de los eslabones, protegiéndolos y optimizando el diseño compacto del sistema.

Durante el proceso de montaje se identificó la necesidad de perforar los tubos de aluminio en algunas zonas, con el fin de ajustar con mayor precisión la posición de los

motores y lograr la tensión óptima en las correas dentadas. Para asegurar una transmisión eficiente y evitar deslizamientos, estas correas deben estar lo más tensas posible, lo que exige un posicionamiento milimétrico de los componentes. Por otro lado, también fue necesario realizar perforaciones laterales adicionales en los tubos para permitir la salida de los cables de alimentación y control de los motores hacia el exterior. Estas aberturas se realizaron con especial cuidado, asegurando un diámetro suficiente para evitar presión sobre los cables y eliminando cualquier borde afilado que pudiera dañarlos durante el funcionamiento prolongado del sistema.

Una vez ensambladas las diferentes piezas y comprobado su correcto encaje, se procedió a pegar con adhesivo epoxi todos los elementos que lo requerían, incluyendo las varillas, los eslabones y las piezas de articulación. Este proceso garantizó una unión firme y duradera, asegurando la rigidez y estabilidad necesarias para el correcto funcionamiento del brazo robótico.



Figura 36: Montaje de la estructura

A continuación, se realizó el atornillado de los motores a las piezas correspondientes, proceso que incluyó varias pruebas y ajustes para optimizar la tensión de las correas dentadas y la rigidez de la estructura. En diversas ocasiones fue necesario reajustar la tensión, ya que cuando las correas no estaban suficientemente tensas, el sistema no lograba sostenerse por sí mismo y presentaba holguras que afectaban al movimiento y precisión del brazo.

Posteriormente, para proporcionar una base estable al conjunto, el brazo fue atornillado a una tabla de madera de dimensiones 60 x 30 cm utilizando seis tornillos M4. Esta

fijación mecánica permite que la estructura permanezca firmemente sujeta, evitando movimientos indeseados durante la operación. Además, esta tabla puede ser fácilmente asegurada a una mesa o superficie de trabajo, facilitando su uso y manipulación en diferentes entornos.



Figura 37: Ensamblado a la tabla de madera

El proceso de montaje del brazo robótico ha sido una fase clave en la materialización del diseño desarrollado previamente en Fusion 360. A lo largo de las distintas etapas constructivas se ha logrado transformar un conjunto de piezas impresas en resina, perfiles de aluminio y componentes mecánicos en una estructura funcional, sólida y operativa.

Desde el corte y mecanizado de los perfiles metálicos hasta el ensamblaje de las articulaciones mediante rodamientos y varillas de acero inoxidable, cada paso ha requerido precisión, ajustes sucesivos y pruebas prácticas para garantizar la correcta alineación, tensión y movilidad del sistema. La incorporación de correas dentadas para la transmisión del movimiento entre los motores y los eslabones ha implicado especial atención al posicionamiento y tensión de las mismas, siendo necesario modificar y adaptar algunas perforaciones para lograr un rendimiento óptimo.

Las uniones críticas se han asegurado mediante adhesivo epoxi, lo que ha contribuido a mantener la rigidez estructural del conjunto. Finalmente, la fijación del brazo a una base de madera ha proporcionado una plataforma estable que permite su manipulación y funcionamiento con seguridad.

En conjunto, el proceso ha demostrado la viabilidad de construir un sistema robótico funcional a partir de recursos limitados y técnicas accesibles, alcanzando resultados satisfactorios tanto en términos estructurales como mecánicos.



Figura 38: Robot ensamblado

8. Software

Descripción de los programas utilizados

Durante el desarrollo del brazo robótico se ha hecho uso de diversas herramientas software que han facilitado tanto el diseño mecánico como el desarrollo electrónico y la programación del sistema. A continuación, se detallan los programas más relevantes empleados a lo largo del proyecto:

Fusion 360

Este software de modelado 3D ha sido utilizado para diseñar de forma completa la estructura mecánica del brazo robótico, incluyendo eslabones, piezas articuladas, soportes para motores y otras partes impresas en resina. Fusion 360 ha permitido realizar simulaciones de movimiento, validar interferencias entre componentes y exportar archivos STL para su posterior impresión 3D. Gracias a su entorno colaborativo, también ha sido posible trabajar en equipo sobre los mismos archivos de forma coordinada.

AutoCAD

AutoCAD se empleó en las fases iniciales del diseño para definir el layout del sistema y plantear las dimensiones generales del entorno de trabajo. También se utilizó para

realizar planos técnicos necesarios para documentar el proyecto, especialmente en lo relacionado con la base y la distribución de componentes sobre la superficie de montaje.

Arduino IDE

El entorno de desarrollo de Arduino ha sido fundamental para la programación del microcontrolador que controla el brazo robótico. Desde este software se han escrito y cargado los programas necesarios para controlar el movimiento de los motores, gestionar entradas digitales y generar señales PWM.

KiCad

KiCad fue el programa utilizado para diseñar la PCB personalizada que integra todos los componentes electrónicos del proyecto. Esta herramienta permitió crear tanto el esquema eléctrico como el diseño físico de la placa, facilitando el ruteado de pistas, la colocación de componentes, la verificación de errores eléctricos y la generación de los archivos Gerber necesarios para su fabricación.

9. Presupuesto

Para la realización del brazo robótico diseñado, se estableció un presupuesto inicial máximo de 200 euros. Este límite económico se definió con el objetivo de desarrollar una solución técnicamente funcional, pero que a su vez fuese viable desde el punto de vista económico, minimizando los costes sin comprometer la calidad ni el rendimiento del prototipo.

Presupuesto inicial:	200		
Gastos	Precio/unidad	Unidades	Precio total
Motores 5 rpm	6,49	2	12,98
Motores 10 rpm	6,49	2	12,98
Poleas de transmision mecanica	11,56	3	34,68
Barra de aluminio	16,78	1	16,78
Barilla de acero inoxidable	0,99	1	0,99
Resina para impresion 3D	25	1	25
Paquete de Cables	13,59	1	13,59
Placa esp32	3,5	1	3,5
Rodamiento 6200	2,46	1	2,46
Rodamiento 606zz	3,99	1	3,99
Rodamiento 683 zz	2,9	1	2,9
TOTAL GASTADO	133,84		
MARGEN DE PRECIO	66,16		

Figura 39: Excel de gastos

En la tabla superior, se muestra el desglose de precios, tras la selección y adquisición de componentes. El coste total incurrido ha sido de 133,84 €, lo que representa un 66,92 % del presupuesto disponible, es decir, un margen de ahorro de 66,16 €.

Es importante destacar que no todos los componentes utilizados en la construcción del prototipo se han incluido en este presupuesto. Algunas piezas y materiales —como elementos de fijación, herramientas, piezas de montaje impresas previamente, o componentes electrónicos comunes— ya se encontraban disponibles en el laboratorio o en propiedad del equipo de trabajo, por lo que no ha sido necesario adquirirlos para este proyecto en concreto.

La ejecución del proyecto ha demostrado una muy buena gestión de los recursos económicos, logrando una solución funcional, robusta y bien dimensionada, utilizando poco más de la mitad del presupuesto disponible. Este resultado refleja un uso racional y eficiente de los recursos, que, además, deja margen económico suficiente para posibles ajustes, mejoras futuras o reposiciones en caso de fallos.

En definitiva, el diseño presentado se configura como una alternativa económica y eficaz dentro del ámbito de la robótica funcional, cumpliendo sobradamente con los requisitos técnicos establecidos y manteniendo al mismo tiempo una estrategia de costes optimizada.

Tras la construcción del primer brazo, hemos comprobado que la mayoría de los componentes adquiridos (resina de impresión 3D, barras de aluminio, varillas de acero inoxidable, poleas, rodamientos, cables y placa ESP32) quedan aún disponibles en cantidad suficiente para fabricar un segundo robot completo. La única excepción son los motores, de los cuales únicamente ha quedado un ejemplar sin usar. Por tanto, para replicar totalmente el diseño haría falta adquirir dos motores adicionales (del mismo tipo y velocidad) para disponer de todos los actuadores necesarios en el nuevo prototipo.

Este sobrante de material no solo demuestra la eficiencia de nuestra gestión de recursos, sino que también añade un valor extra al proyecto, al facilitar la escala y reproducción del sistema en un segundo prototipo con un coste marginal muy bajo.

10. Resultado

A lo largo del desarrollo del proyecto se han obtenido resultados altamente positivos en lo que respecta al diseño estructural, la construcción física del robot y el establecimiento de un modelo matemático funcional. A continuación, se detallan los logros alcanzados y las ventajas que estos aportan al sistema global del brazo robótico.

1. Ventajas derivadas del diseño 3D en Fusion 360

El diseño tridimensional completo del brazo robótico en Autodesk Fusion 360 ha permitido previsualizar y optimizar desde una fase temprana todos los aspectos críticos del sistema mecánico. Se logró una configuración estructural altamente compacta y

funcional gracias a la disposición de los motores en el interior de los eslabones y al uso de perfiles de aluminio de 40x40 mm como base estructural. Esta elección no solo aporta rigidez y ligereza, sino que facilita enormemente la integración modular de los componentes, permitiendo futuras modificaciones o ampliaciones sin rediseño completo.

Además, la visualización digital anticipada de las interferencias, tolerancias y ejes de rotación posibilitó una drástica reducción de errores de montaje y una planificación eficiente de la fabricación de piezas. Esto representa una ventaja significativa en términos de coste y tiempo de producción, y dota al proyecto de una solidez industrial que supera el carácter meramente académico del trabajo.

2. Beneficios estructurales de la arquitectura mecánica

Desde el punto de vista físico, la estructura construida reproduce con precisión el modelo digital, validando tanto las cotas como los ajustes previstos. La inclusión de tres articulaciones rotacionales independientes permite un grado elevado de maniobrabilidad en el espacio tridimensional. La configuración geométrica del brazo —con dos eslabones de longitudes diferentes y libertad rotacional en cada eje— habilita una zona de trabajo extensa en comparación con el volumen del propio robot.

La decisión de usar correas dentadas con motores fijos y ejes intermedios facilita el mantenimiento del centro de masas en posiciones favorables, reduciendo el esfuerzo necesario en los actuadores. Esto contribuye directamente a una mayor estabilidad durante el funcionamiento y, por tanto, a una mejora del rendimiento dinámico general.

Asimismo, el diseño de la base circular giratoria, compuesta por tres capas —soporte, alojamiento de rodamiento y plataforma estructural— proporciona un soporte robusto que amortigua vibraciones y permite una rotación fluida del conjunto. Esta característica mejora tanto la precisión del posicionamiento como la repetibilidad del robot, dos parámetros fundamentales en cualquier sistema automatizado.

3. Ventajas del montaje por fases y fabricación mixta

El sistema de fabricación y montaje empleado, basado en una combinación de mecanizado tradicional e impresión 3D, ha permitido una elevada adaptabilidad y eficiencia en la producción de componentes. Las piezas críticas, como los alojamientos de rodamientos o los soportes de motores, fueron diseñadas a medida e impresas en PLA reforzado, lo que redujo drásticamente los costes sin comprometer la funcionalidad.

La estrategia de montaje por fases permitió validar progresivamente la precisión de ensamblado y el ajuste funcional de cada subcomponente. Gracias a esta metodología, se detectaron y corrigieron pequeños desajustes sin necesidad de rehacer grandes partes del conjunto, demostrando la fiabilidad del diseño 3D como guía de fabricación. Esta capacidad de adaptación representa un resultado especialmente valioso en contextos de prototipado rápido o producción a baja escala.

4. Modelado matemático y validación analítica

Uno de los logros más importantes ha sido el desarrollo de un modelo matemático completo de la cinemática directa e inversa del brazo. Este modelo permite conocer con precisión tanto la posición del extremo del brazo a partir de los ángulos de las articulaciones (cinemática directa), como el cálculo de los ángulos necesarios para alcanzar una posición deseada (cinemática inversa).

En particular, se implementaron funciones que resuelven de forma analítica la cinemática inversa considerando múltiples soluciones (codo hacia arriba y codo hacia abajo), lo cual amplía significativamente la flexibilidad del sistema en su zona de trabajo. Esta capacidad de escoger entre configuraciones alternativas ante un mismo objetivo espacial es una ventaja clave en robótica, ya que permite evitar obstáculos, minimizar esfuerzos o mantener la orientación deseada del efector final.

El modelo desarrollado se programó en lenguaje C, lo que además de garantizar eficiencia computacional, permite su implementación directa en sistemas embebidos y microcontroladores en futuras versiones. Las fórmulas obtenidas están basadas en principios geométricos robustos y han sido estructuradas para facilitar su validación mediante simulaciones numéricas y comparación con la cinemática directa.

En conjunto, los resultados obtenidos en esta primera fase evidencian un diseño estructural sólido, versátil y bien adaptado a los requisitos funcionales del proyecto. La implementación eficaz del diseño 3D, la fabricación modular y el modelado matemático preciso han sentado las bases para una operación exitosa del brazo robótico. En la segunda parte del análisis de resultados, se abordará la validación práctica de la solución mediante pruebas de posicionamiento, simulación de trayectorias y verificación de la precisión funcional del sistema en operación

11. Reparto de tareas

Nombre, Apellidos	TAREAS
Miguel Olalla	• PCB • Programación lógica matemática • Ensamblaje robot • Diseño del robot • Conexiones electrónicas • Montaje
Paula Verdejo	• Conexión electrónica • Programación electrónica (PID) • Programación lógica matemática • Memoria
Alba Martínez	• Aplicación móvil • Programación lógica matemática

	• Memoria
Marcos Rodríguez	• Diseño del robot • Modelo matemático • Programación lógica matemática • Memoria • Ensamblaje robot
Jorge Moya	• Diseño del robot • Modelo matemático • Memoria • Ensamblaje del robot

12. NOTAS

ALUMNO	NOTA
MARCOS RODRIGUEZ	9.6
MIGUEL OLALLA	10
PAULA VERDEJO	9.4
ALBA MARTÍNEZ	9.4
JORGE MOYA	8.5

Reparto de puntos

Reparto de 50 puntos entre los 5 miembros del grupo

ALUMNO	Puntos sobre 50
MARCOS RODRIGUEZ	9
MIGUEL OLALLA	14
PAULA VERDEJO	9
ALBA MARTÍNEZ	9
JORGE MOYA	8

13. CÓDIGO FINAL

```
#include <Arduino.h>
#include <WiFi.h>
#include <math.h>

#define PI 3.14159265358979323846

//Angulos calibración

#define Ang_Base 0
#define Ang_Hombro 90
#define Ang_Codo 0

// ----- WiFi -----
const char* ssid = "ROBOTICO";
const char* password = "12345676";
WiFiServer server(80);

// ----- Escalado de coordenadas desde móvil -----
const float tablero_lado_largo_m = 0.6;
const float pantalla_movil_largo_m = 0.19;
const float escala_x = tablero_lado_largo_m /
pantalla_movil_largo_m;

const float tablero_lado_alto_m = 0.3;
const float pantalla_movil_alto_m = 0.10;
const float escala_y = tablero_lado_alto_m / pantalla_movil_alto_m;

// ----- Altura fija (Z) -----
```

```

#define ALTURA_FIJA 30.0 // mm

//-----

static bool recogidaAsignada = false;
static bool recogidaAlcanzada = false;
static bool excavadoCompletado=false;
static bool puntoMedio = false;
static bool depositoAlcanzado=false;

// ----- Estructura Motor -----
struct Motor {
    const char* name;
    int in1, in2, ena;
    int encA, encB;
    int pwmChannel;
    long pulsesPerTurn;
    float Kp, Ki, Kd;

    volatile long encoderCount = 0;
    float targetDegrees = 0;
    long targetPulses = 0;
    float controlSignal = 0;
    float previousError = 0;
    float errorIntegral = 0;
    unsigned long previousTime = 0;
};

Motor motorBase;
Motor motorHombro;
Motor motorCodo;
Motor motorpala;

// Variables globales para tareas
float x_recibido = 0.0;
float y_recibido = 0.0;
float x_escalado = 0.0;
float y_escalado = 0.0;

bool nuevaTarea = false;
const float x_deposito_fijo = -0.33;
const float y_deposito_fijo = 0.3;

```

```

// ----- Funciones auxiliares -----
double rad_to_deg(double rad) { return rad * 180.0 / PI; }
double deg_to_rad(double deg) { return deg * PI / 180.0; }

// ----- Cinemática inversa -----
bool cinematicaInversa(double x, double y, double z, double L1,
double L2,
    double* theta1, double* theta2, double* theta3) {

    *theta1 = atan2(y, x);
    double R = sqrt(x * x + y * y);
    double D = (R * R + z * z - L1 * L1 - L2 * L2) / (2.0 * L1 * L2);

    // if (D < -1.0 || D > 1.0) return false;

    double theta3_options[2] = {
        atan2(sqrt(1.0 - D * D), D),
        atan2(-sqrt(1.0 - D * D), D)
    };

    for (int i = 0; i < 2; i++) {
        double t3 = theta3_options[i];
        double beta = atan2(z, R);
        double alpha = atan2(L2 * sin(t3), L1 + L2 * cos(t3));
        double t2 = beta - alpha;

        double t1_deg = rad_to_deg(*theta1);
        double t2_deg = rad_to_deg(t2);
        double t3_deg = rad_to_deg(t3);

        if (t1_deg < 0) t1_deg += 360;

        if (t1_deg >= 0 && t1_deg <= 360 &&
            t2_deg >= 30 && t2_deg <= 110 &&
            t3_deg >= -120 && t3_deg <= 30) {

            *theta1 = deg_to_rad(t1_deg); // normalizado
            *theta2 = deg_to_rad(t2_deg);
            *theta3 = deg_to_rad(t3_deg);
            return true;
        }
    }
}

```

```

    return false;
}

//-----PROTOTIPOS PALA-----
void moverPalaAGrados(float gradosObjetivo);

//-----PROTOTIPO MOVIMIENTO-----
void irDeRecogidaADeposito(float x_recogida, float y_recogida, float
x_deposito, float y_deposito);

// ----- Encoder ISR -----
void readEncoderBase() {
    if (digitalRead(motorBase.encB)) motorBase.encoderCount++;
    else motorBase.encoderCount--;
}

void readEncoderHombro() {
    if (digitalRead(motorHombro.encB)) motorHombro.encoderCount++;
    else motorHombro.encoderCount--;
}

void readEncoderCodo() {
    if (digitalRead(motorCodo.encB)) motorCodo.encoderCount++;
    else motorCodo.encoderCount--;
}

void readEncoderpala() {
    if (digitalRead(motorpala.encB)) motorpala.encoderCount++;
    else motorpala.encoderCount--;
}

// ----- PID -----
void updatePID(Motor& m) {
    unsigned long now = micros();
    float dt = (now - m.previousTime) / 1e6;
    m.previousTime = now;

    float error = m.targetPulses - m.encoderCount;
    float dError = (error - m.previousError) / dt;

```

```

float toleranciaGrados = 2.0;
if (fabs(error) < ((m.pulsesPerTurn / 360.0) * toleranciaGrados))
{
    m.controlSignal = 0;
    m.errorIntegral = 0;
    return;
}

m.errorIntegral += error * dt;
m.controlSignal = m.Kp * error + m.Ki * m.errorIntegral + m.Kd *
dError;

m.controlSignal = constrain(m.controlSignal, -255, 255);

m.previousError = error;
}

// ----- Motor driver -----
void driveMotor(Motor& m) {
    int pwm = (int)fabs(m.controlSignal);
    pwm = constrain(pwm, 0, 255);
    int pwmMin = 110; //30
    if (pwm > 0 && pwm < pwmMin) pwm = pwmMin;

    if (m.controlSignal > 0) {
        digitalWrite(m.in1, HIGH); digitalWrite(m.in2, LOW);
    } else if (m.controlSignal < 0) {
        digitalWrite(m.in1, LOW); digitalWrite(m.in2, HIGH);
    } else {
        digitalWrite(m.in1, LOW); digitalWrite(m.in2, LOW);
    }

    ledcWrite(m.pwmChannel, pwm);
}

// ----- Debug Serial -----
void debugPrint() {
    float posBase = ((float)motorBase.encoderCount /
motorBase.pulsesPerTurn) * 360.0;
    float posHombro = ((float)motorHombro.encoderCount /
motorHombro.pulsesPerTurn) * 360.0;
    float posCodo = ((float)motorCodo.encoderCount /
motorCodo.pulsesPerTurn) * 360.0;

```



```

    float pospala = ((float)motorpala.encoderCount /
motorpala.pulsesPerTurn) * 360.0;

    Serial.print("[Base] Objetivo: ");
Serial.print(motorBase.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(posBase);
    Serial.print(" ° | PWM: ");
Serial.println(motorBase.controlSignal);

    Serial.print("[Hombro] Objetivo: ");
Serial.print(motorHombro.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(posHombro);
    Serial.print(" ° | PWM: ");
Serial.println(motorHombro.controlSignal);

    Serial.print("[Codo] Objetivo: ");
Serial.print(motorCodo.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(posCodo);
    Serial.print(" ° | PWM: ");
Serial.println(motorCodo.controlSignal);

    Serial.print("[Pala] Objetivo: ");
Serial.print(motorpala.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(pospala);
    Serial.print(" ° | PWM: ");
Serial.println(motorpala.controlSignal);

    Serial.println();
}

// ----- SETUP -----
void setup()
{

    Serial.begin(115200);

    // Base
    motorBase.name = "Base";
    motorBase.in1 = 10;
    motorBase.in2 = 9;
    motorBase.ena = 11;
    motorBase.encA = 46;
    motorBase.encB = 3;

```

```

motorBase.pwmChannel = 0;
motorBase.pulsesPerTurn = 6600 * 1.5;
motorBase.Kp = 2.0;
motorBase.Ki = 0.00005;
motorBase.Kd = 0.01;

pinMode(motorBase.in1, OUTPUT); pinMode(motorBase.in2, OUTPUT);
pinMode(motorBase.encA, INPUT); pinMode(motorBase.encB, INPUT);
attachInterrupt(digitalPinToInterrupt(motorBase.encA),
readEncoderBase, RISING);

  ledcSetup(motorBase.pwmChannel, 5000, 8);
  ledcAttachPin(motorBase.ena, motorBase.pwmChannel);
  motorBase.previousTime = micros();

// Hombro
motorHombro.name = "Hombro";
motorHombro.in1 = 12;
motorHombro.in2 = 13;
motorHombro.ena = 14;
motorHombro.encA = 47;
motorHombro.encB = 21;
motorHombro.pwmChannel = 1;
motorHombro.pulsesPerTurn = 11 * 900 * 3.75; // 49500
motorHombro.Kp = 1.0;
motorHombro.Ki = 0.00001;
motorHombro.Kd = 0.001;

pinMode(motorHombro.in1, OUTPUT); pinMode(motorHombro.in2,
OUTPUT);
pinMode(motorHombro.encA, INPUT); pinMode(motorHombro.encB,
INPUT);
attachInterrupt(digitalPinToInterrupt(motorHombro.encA),
readEncoderHombro, RISING);
  ledcSetup(motorHombro.pwmChannel, 5000, 8);
  ledcAttachPin(motorHombro.ena, motorHombro.pwmChannel);
  motorHombro.previousTime = micros();

// Codo
motorCodo.name = "Codo";

motorCodo.in1 = 38;
motorCodo.in2 = 37;
motorCodo.ena = 39;

```

```

motorCodo.encA = 36;
motorCodo.encB = 35;
motorCodo.pwmChannel = 2;
motorCodo.pulsesPerTurn = 11 * 900 * 3.75; // 49500
motorCodo.Kp = 1.0;
motorCodo.Ki = 0.00001;
motorCodo.Kd = 0.001;

pinMode(motorCodo.in1, OUTPUT); pinMode(motorCodo.in2, OUTPUT);
pinMode(motorCodo.encA, INPUT); pinMode(motorCodo.encB, INPUT);
attachInterrupt(digitalPinToInterrupt(motorCodo.encA),
readEncoderCodo, RISING);
  ledcSetup(motorCodo.pwmChannel, 5000, 8);
  ledcAttachPin(motorCodo.ena, motorCodo.pwmChannel);
  motorCodo.previousTime = micros();

// Pala

motorpala.name = "pala";

motorpala.in1 = 42;
motorpala.in2 = 41;
motorpala.ena = 40;
motorpala.encA = 1;
motorpala.encB = 2;
motorpala.pwmChannel = 3;
motorpala.pulsesPerTurn = 6600 ;
motorpala.Kp = 1.0;
motorpala.Ki = 0.00005;
motorpala.Kd = 0.01;

pinMode(motorpala.in1, OUTPUT); pinMode(motorpala.in2, OUTPUT);
pinMode(motorpala.encA, INPUT); pinMode(motorpala.encB, INPUT);
attachInterrupt(digitalPinToInterrupt(motorpala.encA),
readEncoderpala, RISING);
  ledcSetup(motorpala.pwmChannel, 5000, 8);
  ledcAttachPin(motorpala.ena, motorpala.pwmChannel);
  motorpala.previousTime = micros();
// WiFi
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
{
  delay(1000);

```

```

    Serial.println("Conectando a WiFi...");
}
Serial.print("Conectado. IP: "); Serial.println(WiFi.localIP());
server.begin();

//INICIALIZACION DE ANGULOS
motorBase.encoderCount = motorBase.pulsesPerTurn*Ang_Base/360;
motorHombro.encoderCount =
motorHombro.pulsesPerTurn*Ang_Hombro/360;
motorCodo.encoderCount = motorCodo.pulsesPerTurn*Ang_Codo/360;

motorBase.targetDegrees = Ang_Base;
motorHombro.targetDegrees = Ang_Hombro;
motorCodo.targetDegrees = Ang_Codo;

//referenciado en vertical el brazo entero
motorBase.targetPulses = motorBase.encoderCount;
motorHombro.targetPulses = motorHombro.encoderCount;
motorCodo.targetPulses = motorCodo.encoderCount;

moverPalaAGrados(0); // Dejamos abierta la pala completamente

}

// ----- LOOP -----
void loop()
{
    updatePID(motorBase);    driveMotor(motorBase);
    updatePID(motorHombro);  driveMotor(motorHombro);
    updatePID(motorCodo);    driveMotor(motorCodo);
    updatePID(motorpala);    driveMotor(motorpala);
    // ----- SERIAL: Ángulos manuales -----
    if (Serial.available()) {
        String entrada = Serial.readStringUntil('\n');
        entrada.trim();

        int bIndex = entrada.indexOf("B:");
        int hIndex = entrada.indexOf(",H:");
        int cIndex = entrada.indexOf(",C:");
        int pIndex = entrada.indexOf(",P:");

        if (bIndex != -1 && hIndex != -1 && cIndex != -1) {
            String bStr = entrada.substring(bIndex + 2, hIndex);

```

```

String hStr = entrada.substring(hIndex + 3, cIndex);
String cStr = entrada.substring(cIndex + 3, pIndex);
String pStr = entrada.substring(pIndex + 3);

motorBase.targetDegrees = bStr.toFloat();
motorHombro.targetDegrees = hStr.toFloat();
motorCodo.targetDegrees = cStr.toFloat();
motorpala.targetDegrees = pStr.toFloat();
Serial.println(pStr.toFloat());

motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;
motorpala.targetPulses = (motorpala.targetDegrees / 360.0) *
motorpala.pulsesPerTurn;

Serial.println("Ángulos por serial recibidos y aplicados.");
} else {
    Serial.println("Formato incorrecto por serial. Usa
B:x,H:y,C:z");
}
}

// ----- WIFI: Coordenadas de la app (cinemática inversa)
-----
WiFiClient client = server.available();
if (client) {
    Serial.println("Cliente conectado.");
    while (client.connected()) {
        if (client.available()) {
            String mensaje = client.readStringUntil('\n');
            mensaje.trim();
            client.println("OK");
            Serial.print("Recibido: ");
            Serial.println(mensaje);

            int xIndex = mensaje.indexOf("X:");
            int yIndex = mensaje.indexOf(",Y:");
            if (xIndex != -1 && yIndex != -1) {
                String xStr = mensaje.substring(xIndex + 2, yIndex);

```

```

String yStr = mensaje.substring(yIndex + 3);

float x_m = xStr.toFloat();
float y_m = yStr.toFloat();

x_escalado = x_m * escala_x;
y_escalado = y_m * escala_y;

// Guardar la nueva tarea de recogida
nuevaTarea = true;
recogidaAsignada = true;

client.println("OK");
} else {
    client.println("Formato inválido");
}
}
}
client.stop();
Serial.println("Cliente desconectado.");
}

// Ejecutar secuencia de recogida → depósito si hay nueva tarea
if (nuevaTarea)
{
    Serial.println("tarea nueva");
    irDeRecogidaADeposito(x_escalado, y_escalado, x_deposito_fijo,
y_deposito_fijo);

}

debugPrint();
delay(50);
}

//FUNCIÓN MOVIMIENTO DEL MOTOR A RECOGIDA Y DEPÓSITO
void irDeRecogidaADeposito(float x1, float y1, float x_deposito,
float y_deposito)
{
    Serial.println("aqui");

```



```

double t0, t1, t2;

const float z = ALTURA_FIJA;
const double L1 = 275.0 + 43.0;
const double L2 = 280.0;
float umbral = 100.0;

float x2 = x_deposito;
float y2 = y_deposito;

// 1. Asignar posición de recogida

if (recogidaAsignada)
{
    Serial.println("recogidaAsignada");
    if (cinematicaInversa(x1 * 1000.0, y1 * 1000.0, ALTURA_FIJA-20,
L1, L2, &t0, &t1, &t2)) {
        motorBase.targetDegrees = rad_to_deg(t0);
        motorHombro.targetDegrees = rad_to_deg(t1);
        motorCodo.targetDegrees = rad_to_deg(t2);

        motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);
        updatePID(motorHombro);  driveMotor(motorHombro);
        updatePID(motorCodo);    driveMotor(motorCodo);

    }

// 2. Comprobar si ha llegado al punto de recogida

bool baseOK    = fabs(motorBase.controlSignal) < umbral;
bool hombroOK  = fabs(motorHombro.controlSignal) < umbral;
bool codoOK    = fabs(motorCodo.controlSignal) < umbral;

```

```

bool palaOK    = fabs(motorpala.controlSignal) < umbral;

if (baseOK && hombroOK && codoOK && palaOK)
{
    Serial.println("Ha llegado al punto de recogida");
    //
    delay(500);
    recogidaAlcanzada = true;
    recogidaAsignada = false;
    Serial.println("Recogida alcanzada=true");
}

}

//Mover directamente abajo
if(recogidaAlcanzada)
{
    Serial.println("recogidaAlcanzada");

    if (cinematicaInversa(x1 * 1000.0, y1 * 1000.0 , -50, L1, L2,
&t0, &t1, &t2)) {
        motorBase.targetDegrees = rad_to_deg(t0);
        motorHombro.targetDegrees = rad_to_deg(t1);
        motorCodo.targetDegrees = rad_to_deg(t2);

        motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);
        updatePID(motorHombro);  driveMotor(motorHombro);
        updatePID(motorCodo);    driveMotor(motorCodo);
        delay(0.1);
        moverPalaAGrados(-100);  // cerrar pala

    }

    bool baseOK    = fabs(motorBase.controlSignal) < umbral;
    bool hombroOK = fabs(motorHombro.controlSignal) < umbral;

```

```

bool codoOK    = fabs(motorCodo.controlSignal) < umbral;
bool palaOK    = fabs(motorpala.controlSignal) < umbral;

    if(baseOK && hombroOK && codoOK && palaOK)
    {

        excavadoCompletado =true;
        recogidaAlcanzada = false;
    }
}

if(excavadoCompletado)
{
    Serial.println("excavadoCompletado");
    //Se mueve a posicion directamente arriba
    if (cinematicaInversa(x1 * 1000.0, y1 * 1000.0,
ALTURA_FIJA+80, L1, L2, &t0, &t1, &t2)) {
        motorBase.targetDegrees = rad_to_deg(t0);
        motorHombro.targetDegrees = rad_to_deg(t1);
        motorCodo.targetDegrees = rad_to_deg(t2);

        motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);
        updatePID(motorHombro);  driveMotor(motorHombro);
        updatePID(motorCodo);    driveMotor(motorCodo);

    }

bool baseOK    = fabs(motorBase.controlSignal) < umbral;
bool hombroOK  = fabs(motorHombro.controlSignal) < umbral;
bool codoOK    = fabs(motorCodo.controlSignal) < umbral;
bool palaOK    = fabs(motorpala.controlSignal) < umbral;

if(baseOK && hombroOK && codoOK && palaOK)

```

```

{
    excavadoCompletado =false;
    puntoMedio=true;
}
}

// 4. Ir a la zona de deposito
if (puntoMedio)
{
    Serial.println("puntoMedio");
    if (cinematicaInversa(x2 * 1000.0, y2 * 1000.0, ALTURA_FIJA+60,
L1, L2, &t0, &t1, &t2)) {
        motorBase.targetDegrees = rad_to_deg(t0);
        motorHombro.targetDegrees = rad_to_deg(t1);
        motorCodo.targetDegrees = rad_to_deg(t2);

        motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);
        updatePID(motorHombro);  driveMotor(motorHombro);
        updatePID(motorCodo);    driveMotor(motorCodo);

    }

    bool baseOK    = fabs(motorBase.controlSignal) < umbral;
    bool hombroOK  = fabs(motorHombro.controlSignal) < umbral;
    bool codoOK    = fabs(motorCodo.controlSignal) < umbral;
    bool palaOK    = fabs(motorpala.controlSignal) < umbral;

    if (baseOK && hombroOK && codoOK && palaOK)
    {
        Serial.println("Ha llegado al punto de deposito");
        puntoMedio =false;
        depositoAlcanzado=true;
        delay(500);
    }
}

```

```

    }

    // 4. Cuando llega al depósito → abrir la pala y reiniciar
    if (depositoAlcanzado)
    {
        Serial.println("depositoAlcanzado");
        moverPalaAGrados(50); // abrir la pala
        delay(500);           // espera para soltar la arena

        // Reiniciar el ciclo de tareas
        nuevaTarea = false;
        recogidaAsignada = false;
        recogidaAlcanzada = false;
        excavadoCompletado = false;
        puntoMedio = false;
        depositoAlcanzado = false;
    }
}

void moverPalaAGrados(float gradosObjetivo)
{
    motorpala.targetDegrees = gradosObjetivo;
    motorpala.targetPulses = (motorpala.targetDegrees / 360.0) *
motorpala.pulsesPerTurn;

    updatePID(motorpala);
    driveMotor(motorpala);
}

#include <Arduino.h>
#include <WiFi.h>
#include <math.h>

#define PI 3.14159265358979323846

//Angulos calibración

#define Ang_Base 0
#define Ang_Hombro 90
#define Ang_Codo 0

```

```

// ----- WiFi -----
const char* ssid = "ROBOTICO";
const char* password = "12345676";
WiFiServer server(80);

// ----- Escalado de coordenadas desde móvil -----
const float tablero_lado_largo_m = 0.6;
const float pantalla_movil_largo_m = 0.19;
const float escala_x = tablero_lado_largo_m /
pantalla_movil_largo_m;

const float tablero_lado_alto_m = 0.3;
const float pantalla_movil_alto_m = 0.10;
const float escala_y = tablero_lado_alto_m / pantalla_movil_alto_m;

// ----- Altura fija (Z) -----
#define ALTURA_FIJA 30.0 // mm

//-----

static bool recogidaAsignada = false;
static bool recogidaAlcanzada = false;
static bool excavadoCompletado=false;
static bool puntoMedio = false;
static bool depositoAlcanzado=false;

// ----- Estructura Motor -----
struct Motor {
    const char* name;
    int in1, in2, ena;
    int encA, encB;
    int pwmChannel;
    long pulsesPerTurn;
    float Kp, Ki, Kd;

    volatile long encoderCount = 0;
    float targetDegrees = 0;
    long targetPulses = 0;
    float controlSignal = 0;
    float previousError = 0;

```



```

float errorIntegral = 0;
unsigned long previousTime = 0;
};

Motor motorBase;
Motor motorHombro;
Motor motorCodo;
Motor motorpala;

// Variables globales para tareas
float x_recibido = 0.0;
float y_recibido = 0.0;
float x_escalado = 0.0;
float y_escalado = 0.0;

bool nuevaTarea = false;
const float x_deposito_fijo = -0.33;
const float y_deposito_fijo = 0.3;

// ----- Funciones auxiliares -----
double rad_to_deg(double rad) { return rad * 180.0 / PI; }
double deg_to_rad(double deg) { return deg * PI / 180.0; }

// ----- Cinemática inversa -----
bool cinematicaInversa(double x, double y, double z, double L1,
double L2,
double* theta1, double* theta2, double* theta3) {

    *theta1 = atan2(y, x);
    double R = sqrt(x * x + y * y);
    double D = (R * R + z * z - L1 * L1 - L2 * L2) / (2.0 * L1 * L2);

    // if (D < -1.0 || D > 1.0) return false;

    double theta3_options[2] = {
        atan2(sqrt(1.0 - D * D), D),
        atan2(-sqrt(1.0 - D * D), D)
    };

    for (int i = 0; i < 2; i++) {
        double t3 = theta3_options[i];
        double beta = atan2(z, R);
        double alpha = atan2(L2 * sin(t3), L1 + L2 * cos(t3));
    }
}

```

```

double t2 = beta - alpha;

double t1_deg = rad_to_deg(*theta1);
double t2_deg = rad_to_deg(t2);
double t3_deg = rad_to_deg(t3);

if (t1_deg < 0) t1_deg += 360;

if (t1_deg >= 0 && t1_deg <= 360 &&
    t2_deg >= 30 && t2_deg <= 110 &&
    t3_deg >= -120 && t3_deg <= 30) {

    *theta1 = deg_to_rad(t1_deg); // normalizado
    *theta2 = deg_to_rad(t2_deg);
    *theta3 = deg_to_rad(t3_deg);
    return true;
}

return false;
}

//-----PROTOTIPOS PALA-----
void moverPalaAGrados(float gradosObjetivo);

//-----PROTOTIPO MOVIMIENTO-----
void irDeRecogidaADeposito(float x_recogida, float y_recogida, float
x_deposito, float y_deposito);

// ----- Encoder ISR -----
void readEncoderBase() {
    if (digitalRead(motorBase.encB)) motorBase.encoderCount++;
    else motorBase.encoderCount--;
}

void readEncoderHombro() {
    if (digitalRead(motorHombro.encB)) motorHombro.encoderCount++;
    else motorHombro.encoderCount--;
}

void readEncoderCodo() {

```

```

    if (digitalRead(motorCodo.encB)) motorCodo.encoderCount++;
    else motorCodo.encoderCount--;
}

void readEncoderpala() {
    if (digitalRead(motorpala.encB)) motorpala.encoderCount++;
    else motorpala.encoderCount--;
}

// ----- PID -----
void updatePID(Motor& m) {
    unsigned long now = micros();
    float dt = (now - m.previousTime) / 1e6;
    m.previousTime = now;

    float error = m.targetPulses - m.encoderCount;
    float dError = (error - m.previousError) / dt;

    float toleranciaGrados = 2.0;
    if (fabs(error) < ((m.pulsesPerTurn / 360.0) * toleranciaGrados))
    {
        m.controlSignal = 0;
        m.errorIntegral = 0;
        return;
    }

    m.errorIntegral += error * dt;
    m.controlSignal = m.Kp * error + m.Ki * m.errorIntegral + m.Kd *
dError;

    m.controlSignal = constrain(m.controlSignal, -255, 255);

    m.previousError = error;
}

// ----- Motor driver -----
void driveMotor(Motor& m) {
    int pwm = (int)fabs(m.controlSignal);
    pwm = constrain(pwm, 0, 255);
    int pwmMin = 110; //30
    if (pwm > 0 && pwm < pwmMin) pwm = pwmMin;

    if (m.controlSignal > 0) {

```

```

    digitalWrite(m.in1, HIGH); digitalWrite(m.in2, LOW);
} else if (m.controlSignal < 0) {
    digitalWrite(m.in1, LOW); digitalWrite(m.in2, HIGH);
} else {
    digitalWrite(m.in1, LOW); digitalWrite(m.in2, LOW);
}

    ledcWrite(m.pwmChannel, pwm);
}

// ----- Debug Serial -----
void debugPrint() {
    float posBase = ((float)motorBase.encoderCount /
motorBase.pulsesPerTurn) * 360.0;
    float posHombro = ((float)motorHombro.encoderCount /
motorHombro.pulsesPerTurn) * 360.0;
    float posCodo = ((float)motorCodo.encoderCount /
motorCodo.pulsesPerTurn) * 360.0;
    float pospala = ((float)motorpala.encoderCount /
motorpala.pulsesPerTurn) * 360.0;

    Serial.print("[Base] Objetivo: ");
    Serial.print(motorBase.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(posBase);
    Serial.print(" ° | PWM: ");
    Serial.println(motorBase.controlSignal);

    Serial.print("[Hombro] Objetivo: ");
    Serial.print(motorHombro.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(posHombro);
    Serial.print(" ° | PWM: ");
    Serial.println(motorHombro.controlSignal);

    Serial.print("[Codo] Objetivo: ");
    Serial.print(motorCodo.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(posCodo);
    Serial.print(" ° | PWM: ");
    Serial.println(motorCodo.controlSignal);

    Serial.print("[Pala] Objetivo: ");
    Serial.print(motorpala.targetDegrees);
    Serial.print(" ° | Actual: "); Serial.print(pospala);

```

```

    Serial.print(" ° | PWM: ");
    Serial.println(motorpala.controlSignal);

    Serial.println();
}

// ----- SETUP -----
void setup()
{

    Serial.begin(115200);

    // Base
    motorBase.name = "Base";
    motorBase.in1 = 10;
    motorBase.in2 = 9;
    motorBase.ena = 11;
    motorBase.encA = 46;
    motorBase.encB = 3;
    motorBase.pwmChannel = 0;
    motorBase.pulsesPerTurn = 6600 * 1.5;
    motorBase.Kp = 2.0;
    motorBase.Ki = 0.00005;
    motorBase.Kd = 0.01;

    pinMode(motorBase.in1, OUTPUT); pinMode(motorBase.in2, OUTPUT);
    pinMode(motorBase.encA, INPUT); pinMode(motorBase.encB, INPUT);
    attachInterrupt(digitalPinToInterrupt(motorBase.encA),
readEncoderBase, RISING);
    ledcSetup(motorBase.pwmChannel, 5000, 8);
    ledcAttachPin(motorBase.ena, motorBase.pwmChannel);
    motorBase.previousTime = micros();

    // Hombro
    motorHombro.name = "Hombro";
    motorHombro.in1 = 12;
    motorHombro.in2 = 13;
    motorHombro.ena = 14;
    motorHombro.encA = 47;
    motorHombro.encB = 21;
    motorHombro.pwmChannel = 1;
    motorHombro.pulsesPerTurn = 11 * 900 * 3.75; // 49500
    motorHombro.Kp = 1.0;

```

```

motorHombro.Ki = 0.00001;
motorHombro.Kd = 0.001;

pinMode(motorHombro.in1, OUTPUT); pinMode(motorHombro.in2,
OUTPUT);
pinMode(motorHombro.encA, INPUT); pinMode(motorHombro.encB,
INPUT);
attachInterrupt(digitalPinToInterrupt(motorHombro.encA),
readEncoderHombro, RISING);
ledcSetup(motorHombro.pwmChannel, 5000, 8);
ledcAttachPin(motorHombro.ena, motorHombro.pwmChannel);
motorHombro.previousTime = micros();

// Codo
motorCodo.name = "Codo";

motorCodo.in1 = 38;
motorCodo.in2 = 37;
motorCodo.ena = 39;
motorCodo.encA = 36;
motorCodo.encB = 35;
motorCodo.pwmChannel = 2;
motorCodo.pulsesPerTurn = 11 * 900 * 3.75; // 49500
motorCodo.Kp = 1.0;
motorCodo.Ki = 0.00001;
motorCodo.Kd = 0.001;

pinMode(motorCodo.in1, OUTPUT); pinMode(motorCodo.in2, OUTPUT);
pinMode(motorCodo.encA, INPUT); pinMode(motorCodo.encB, INPUT);
attachInterrupt(digitalPinToInterrupt(motorCodo.encA),
readEncoderCodo, RISING);
ledcSetup(motorCodo.pwmChannel, 5000, 8);
ledcAttachPin(motorCodo.ena, motorCodo.pwmChannel);
motorCodo.previousTime = micros();

// Pala
motorpala.name = "pala";

motorpala.in1 = 42;
motorpala.in2 = 41;
motorpala.ena = 40;
motorpala.encA = 1;

```

```

    motorpala.encB = 2;
    motorpala.pwmChannel = 3;
    motorpala.pulsesPerTurn = 6600 ;
    motorpala.Kp = 1.0;
    motorpala.Ki = 0.00005;
    motorpala.Kd = 0.01;

    pinMode(motorpala.in1, OUTPUT); pinMode(motorpala.in2, OUTPUT);
    pinMode(motorpala.encA, INPUT); pinMode(motorpala.encB, INPUT);
    attachInterrupt(digitalPinToInterrupt(motorpala.encA),
readEncoderpala, RISING);
    ledcSetup(motorpala.pwmChannel, 5000, 8);
    ledcAttachPin(motorpala.ena, motorpala.pwmChannel);
    motorpala.previousTime = micros();
    // WiFi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED)
    {
        delay(1000);
        Serial.println("Conectando a WiFi...");
    }
    Serial.print("Conectado. IP: "); Serial.println(WiFi.localIP());
    server.begin();

    //INICIALIZACION DE ANGULOS
    motorBase.encoderCount = motorBase.pulsesPerTurn*Ang_Base/360;
    motorHombro.encoderCount =
motorHombro.pulsesPerTurn*Ang_Hombro/360;
    motorCodo.encoderCount = motorCodo.pulsesPerTurn*Ang_Codo/360;

    motorBase.targetDegrees = Ang_Base;
    motorHombro.targetDegrees = Ang_Hombro;
    motorCodo.targetDegrees = Ang_Codo;

    //referenciado en vertical el brazo entero
    motorBase.targetPulses = motorBase.encoderCount;
    motorHombro.targetPulses = motorHombro.encoderCount;
    motorCodo.targetPulses = motorCodo.encoderCount;

    moverPalaAGrados(0); // Dejamos abierta la pala completamente
}

```



```
// ----- LOOP -----
void loop()
{
    updatePID(motorBase);    driveMotor(motorBase);
    updatePID(motorHombro);  driveMotor(motorHombro);
    updatePID(motorCodo);    driveMotor(motorCodo);
    updatePID(motorpala);    driveMotor(motorpala);
    // ----- SERIAL: Ángulos manuales -----
    if (Serial.available()) {
        String entrada = Serial.readStringUntil('\n');
        entrada.trim();

        int bIndex = entrada.indexOf("B:");
        int hIndex = entrada.indexOf(",H:");
        int cIndex = entrada.indexOf(",C:");
        int pIndex = entrada.indexOf(",P:");

        if (bIndex != -1 && hIndex != -1 && cIndex != -1) {
            String bStr = entrada.substring(bIndex + 2, hIndex);
            String hStr = entrada.substring(hIndex + 3, cIndex);
            String cStr = entrada.substring(cIndex + 3, pIndex);
            String pStr = entrada.substring(pIndex + 3);

            motorBase.targetDegrees = bStr.toFloat();
            motorHombro.targetDegrees = hStr.toFloat();
            motorCodo.targetDegrees = cStr.toFloat();
            motorpala.targetDegrees = pStr.toFloat();
            Serial.println(pStr.toFloat());

            motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
            motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
            motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;
            motorpala.targetPulses = (motorpala.targetDegrees / 360.0) *
motorpala.pulsesPerTurn;

            Serial.println("Ángulos por serial recibidos y aplicados.");
        } else {
            Serial.println("Formato incorrecto por serial. Usa
B:x,H:y,C:z");
        }
    }
}
```

```

    }

    // ----- WIFI: Coordenadas de la app (cinemática inversa)
    -----
    WiFiClient client = server.available();
    if (client) {
        Serial.println("Cliente conectado.");
        while (client.connected()) {
            if (client.available()) {
                String mensaje = client.readStringUntil('\n');
                mensaje.trim();
                client.println("OK");
                Serial.print("Recibido: ");
                Serial.println(mensaje);

                int xIndex = mensaje.indexOf("X:");
                int yIndex = mensaje.indexOf(",Y:");
                if (xIndex != -1 && yIndex != -1) {
                    String xStr = mensaje.substring(xIndex + 2, yIndex);
                    String yStr = mensaje.substring(yIndex + 3);

                    float x_m = xStr.toFloat();
                    float y_m = yStr.toFloat();

                    x_escalado = x_m * escala_x;
                    y_escalado = y_m * escala_y;

                    // Guardar la nueva tarea de recogida
                    nuevaTarea = true;
                    recogidaAsignada = true;

                    client.println("OK");
                } else {
                    client.println("Formato inválido");
                }
            }
        }
        client.stop();
        Serial.println("Cliente desconectado.");
    }

    // Ejecutar secuencia de recogida → depósito si hay nueva tarea

```

```

    if (nuevaTarea)
    {
        Serial.println("tarea nueva");
        irDeRecogidaADeposito(x_escalado, y_escalado, x_deposito_fijo,
y_deposito_fijo);

    }

    debugPrint();
    delay(50);
}

//FUNCIÓN MOVIMIENTO DEL MOTOR A RECOGIDA Y DEPÓSITO
void irDeRecogidaADeposito(float x1, float y1, float x_deposito,
float y_deposito)
{
    Serial.println("aqui");
    double t0, t1, t2;

    const float z = ALTURA_FIJA;
    const double L1 = 275.0 + 43.0;
    const double L2 = 280.0;
    float umbral = 100.0;

    float x2 = x_deposito;
    float y2 = y_deposito;

    // 1. Asignar posición de recogida

    if (recogidaAsignada)
    {
        Serial.println("recogidaAsignada");
        if (cinematicaInversa(x1 * 1000.0, y1 * 1000.0, ALTURA_FIJA-20,
L1, L2, &t0, &t1, &t2)) {
            motorBase.targetDegrees = rad_to_deg(t0);
            motorHombro.targetDegrees = rad_to_deg(t1);
            motorCodo.targetDegrees = rad_to_deg(t2);

            motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;

```

```

        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);
        updatePID(motorHombro);  driveMotor(motorHombro);
        updatePID(motorCodo);    driveMotor(motorCodo);

    }

// 2. Comprobar si ha llegado al punto de recogida

bool baseOK    = fabs(motorBase.controlSignal) < umbral;
bool hombroOK  = fabs(motorHombro.controlSignal) < umbral;
bool codoOK    = fabs(motorCodo.controlSignal) < umbral;
bool palaOK    = fabs(motorpala.controlSignal) < umbral;

if (baseOK && hombroOK && codoOK && palaOK)
{
    Serial.println("Ha llegado al punto de recogida");
    //
    delay(500);
    recogidaAlcanzada = true;
    recogidaAsignada = false;
    Serial.println("Recogida alcanzada=true");
}

}

//Mover directamente abajo
if(recogidaAlcanzada)
{
    Serial.println("recogidaAlcanzada");

    if (cinematicaInversa(x1 * 1000.0, y1 * 1000.0 , -50, L1, L2,
&t0, &t1, &t2)) {
        motorBase.targetDegrees = rad_to_deg(t0);
        motorHombro.targetDegrees = rad_to_deg(t1);
        motorCodo.targetDegrees = rad_to_deg(t2);
    }
}

```

```

        motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);
        updatePID(motorHombro);  driveMotor(motorHombro);
        updatePID(motorCodo);    driveMotor(motorCodo);
        delay(0.1);
        moverPalaAGrados(-100);  // cerrar pala

    }

    bool baseOK    = fabs(motorBase.controlSignal) < umbral;
    bool hombroOK  = fabs(motorHombro.controlSignal) < umbral;
    bool codoOK    = fabs(motorCodo.controlSignal) < umbral;
    bool palaOK    = fabs(motorpala.controlSignal) < umbral;

    if(baseOK && hombroOK && codoOK && palaOK)
    {

        excavadoCompletado =true;
        recogidaAlcanzada = false;
    }
}

if(excavadoCompletado)
{
    Serial.println("excavadoCompletado");
    //Se mueve a posicion directamente arriba
    if (cinematicaInversa(x1 * 1000.0, y1 * 1000.0,
ALTURA_FIJA+80, L1, L2, &t0, &t1, &t2)) {
        motorBase.targetDegrees = rad_to_deg(t0);
        motorHombro.targetDegrees = rad_to_deg(t1);
        motorCodo.targetDegrees = rad_to_deg(t2);

        motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;

```

```

        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);
        updatePID(motorHombro);  driveMotor(motorHombro);
        updatePID(motorCodo);    driveMotor(motorCodo);

    }

    bool baseOK    = fabs(motorBase.controlSignal) < umbral;
    bool hombroOK  = fabs(motorHombro.controlSignal) < umbral;
    bool codoOK    = fabs(motorCodo.controlSignal) < umbral;
    bool palaOK    = fabs(motorpala.controlSignal) < umbral;

    if(baseOK && hombroOK && codoOK && palaOK)
    {
        excavadoCompletado =false;
        puntoMedio=true;
    }
}

// 4. Ir a la zona de deposito
if (puntoMedio)
{
    Serial.println("puntoMedio");
    if (cinematicaInversa(x2 * 1000.0, y2 * 1000.0, ALTURA_FIJA+60,
L1, L2, &t0, &t1, &t2)) {
        motorBase.targetDegrees = rad_to_deg(t0);
        motorHombro.targetDegrees = rad_to_deg(t1);
        motorCodo.targetDegrees = rad_to_deg(t2);

        motorBase.targetPulses = (motorBase.targetDegrees / 360.0) *
motorBase.pulsesPerTurn;
        motorHombro.targetPulses = (motorHombro.targetDegrees / 360.0)
* motorHombro.pulsesPerTurn;
        motorCodo.targetPulses = (motorCodo.targetDegrees / 360.0) *
motorCodo.pulsesPerTurn;

        updatePID(motorBase);    driveMotor(motorBase);

```

```

    updatePID(motorHombro); driveMotor(motorHombro);
    updatePID(motorCodo);    driveMotor(motorCodo);

}

bool baseOK    = fabs(motorBase.controlSignal) < umbral;
bool hombroOK  = fabs(motorHombro.controlSignal) < umbral;
bool codoOK    = fabs(motorCodo.controlSignal) < umbral;
bool palaOK    = fabs(motorpala.controlSignal) < umbral;

if (baseOK && hombroOK && codoOK && palaOK)
{
    Serial.println("Ha llegado al punto de deposito");
    puntoMedio =false;
    depositoAlcanzado=true;
    delay(500);
}
}

// 4. Cuando llega al depósito → abrir la pala y reiniciar
if (depositoAlcanzado)
{
    Serial.println("depositoAlcanzado");
    moverPalaAGrados(50); // abrir la pala
    delay(500);           // espera para soltar la arena

    // Reiniciar el ciclo de tareas
    nuevaTarea = false;
    recogidaAsignada = false;
    recogidaAlcanzada = false;
    excavadoCompletado = false;
    puntoMedio = false;
    depositoAlcanzado = false;
}
}

void moverPalaAGrados(float gradosObjetivo)
{
    motorpala.targetDegrees = gradosObjetivo;
}

```



```
    motorpala.targetPulses = (motorpala.targetDegrees / 360.0) *  
motorpala.pulsesPerTurn;  
  
    updatePID(motorpala);  
    driveMotor(motorpala);  
}
```